时空数据挖掘

第二讲-数据挖掘与机器学习

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 10 月 22 日

1. 数据挖掘与机器学习的基本概念
2. 数据挖掘的数学基础
3. 特征工程与预处理
4. 传统机器学习模型

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型

数据挖掘

数据挖掘（KDD）定义与流程

- 知识发现与数据挖掘（**Knowledge Discovery and Data Mining**）

数据挖掘（KDD）定义与流程

- **知识发现与数据挖掘（Knowledge Discovery and Data Mining）**
- **定义**：从大量数据中发现新的、有价值的信息和知识的过程

数据挖掘

数据挖掘（KDD）定义与流程

- **知识发现与数据挖掘（Knowledge Discovery and Data Mining）**
- **定义**：从大量数据中发现新的、有价值的信息和知识的过程
- **流程**：

数据挖掘（KDD）定义与流程

- **知识发现与数据挖掘（Knowledge Discovery and Data Mining）**
- **定义**：从大量数据中发现新的、有价值的信息和知识的过程
- **流程**：
  - 数据收集 → 数据清洗 → 数据探索 → 模型构建 → 结果评估

机器学习

机器学习的类型

机器学习

机器学习的类型

- **监督学习（Supervised Learning）**：使用标注数据（Labeled Data）训练模型。

机器学习

机器学习的类型

- **监督学习（Supervised Learning）**：使用标注数据（Labeled Data）训练模型。
- **无监督学习（Unsupervised Learning）**：从无标注数据（Unlabeled Data）中发现模式。

机器学习

机器学习的类型

- **监督学习（Supervised Learning）**：使用标注数据（Labeled Data）训练模型。
- **无监督学习（Unsupervised Learning）**：从无标注数据（Unlabeled Data）中发现模式。
- **半监督学习（Semi-supervised Learning）**：结合少量标注数据和大量无标注数据。

机器学习

机器学习的类型

- **监督学习（Supervised Learning）**：使用标注数据（Labeled Data）训练模型。
- **无监督学习（Unsupervised Learning）**：从无标注数据（Unlabeled Data）中发现模式。
- **半监督学习（Semi-supervised Learning）**：结合少量标注数据和大量无标注数据。
- **强化学习（Reinforcement Learning）**：通过奖励惩罚机制学习最优策略。

机器学习

机器学习的类型

- **监督学习（Supervised Learning）**：使用标注数据（Labeled Data）训练模型。
- **无监督学习（Unsupervised Learning）**：从无标注数据（Unlabeled Data）中发现模式。
- **半监督学习（Semi-supervised Learning）**：结合少量标注数据和大量无标注数据。
- **强化学习（Reinforcement Learning）**：通过奖励惩罚机制学习最优策略。

机器学习的核心作用

机器学习

机器学习的类型

- **监督学习（Supervised Learning）**：使用标注数据（Labeled Data）训练模型。
- **无监督学习（Unsupervised Learning）**：从无标注数据（Unlabeled Data）中发现模式。
- **半监督学习（Semi-supervised Learning）**：结合少量标注数据和大量无标注数据。
- **强化学习（Reinforcement Learning）**：通过奖励惩罚机制学习最优策略。

机器学习的核心作用

- 提供自动化模式发现和预测分析能力。

机器学习

机器学习的类型

- **监督学习（Supervised Learning）**：使用标注数据（Labeled Data）训练模型。
- **无监督学习（Unsupervised Learning）**：从无标注数据（Unlabeled Data）中发现模式。
- **半监督学习（Semi-supervised Learning）**：结合少量标注数据和大量无标注数据。
- **强化学习（Reinforcement Learning）**：通过奖励惩罚机制学习最优策略。

机器学习的核心作用

- 提供自动化模式发现和预测分析能力。
- 处理数据中的复杂依赖关系和非线性特征。

机器学习

机器学习的类型

- **监督学习（Supervised Learning）**：使用标注数据（Labeled Data）训练模型。
- **无监督学习（Unsupervised Learning）**：从无标注数据（Unlabeled Data）中发现模式。
- **半监督学习（Semi-supervised Learning）**：结合少量标注数据和大量无标注数据。
- **强化学习（Reinforcement Learning）**：通过奖励惩罚机制学习最优策略。

机器学习的核心作用

- 提供自动化模式发现和预测分析能力。
- 处理数据中的复杂依赖关系和非线性特征。
- 支持智能决策和知识发现。

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型

概述

为什么需要数学基础?

数据挖掘是从数据中提取信息和知识的过程，而这些数据往往具有随机性和不确定性。而数学提供了处理这种不确定性的方法和工具。

概述

为什么需要数学基础?

数据挖掘是从数据中提取信息和知识的过程，而这些数据往往具有随机性和不确定性。而数学提供了处理这种不确定性的方法和工具。

相关数学课程

- 概率论与数理统计：处理随机性和不确定性

概述

为什么需要数学基础?

数据挖掘是从数据中提取信息和知识的过程，而这些数据往往具有随机性和不确定性。而数学提供了处理这种不确定性的方法和工具。

相关数学课程

- 概率论与数理统计：处理随机性和不确定性
- 线性代数：处理多维数据和矩阵运算

概述

为什么需要数学基础?

数据挖掘是从数据中提取信息和知识的过程，而这些数据往往具有随机性和不确定性。而数学提供了处理这种不确定性的方法和工具。

相关数学课程

- 概率论与数理统计：处理随机性和不确定性
- 线性代数：处理多维数据和矩阵运算
- 优化理论：求解最优化问题和模型训练

概述

为什么需要数学基础?

数据挖掘是从数据中提取信息和知识的过程，而这些数据往往具有随机性和不确定性。而数学提供了处理这种不确定性的方法和工具。

相关数学课程

- 概率论与数理统计：处理随机性和不确定性
- 线性代数：处理多维数据和矩阵运算
- 优化理论：求解最优化问题和模型训练
- 微积分：描述变化和优化问题

概率论与数理统计

核心概念

- 描述性统计：均值、中位数、方差、标准差

概率论与数理统计

核心概念

- 描述性统计：均值、中位数、方差、标准差
- 概率分布：二项分布、正态分布、泊松分布

概率论与数理统计

核心概念

- 描述性统计：均值、中位数、方差、标准差
- 概率分布：二项分布、正态分布、泊松分布
- 条件概率与贝叶斯定理

概率论与数理统计

核心概念

- 描述性统计：均值、中位数、方差、标准差
- 概率分布：二项分布、正态分布、泊松分布
- 条件概率与贝叶斯定理

贝叶斯定理

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

其中：

- $P(A|B)$：后验概率
- $P(B|A)$：似然度
- $P(A)$：先验概率
- $P(B)$：证据

概率论与数理统计

核心概念

- 描述性统计：均值、中位数、方差、标准差
- 概率分布：二项分布、正态分布、泊松分布
- 条件概率与贝叶斯定理
- 假设检验：t 检验、卡方检验

贝叶斯定理

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

其中：

- $P(A|B)$：后验概率
- $P(B|A)$：似然度
- $P(A)$：先验概率
- $P(B)$：证据

概率论与数理统计

核心概念

- 描述性统计：均值、中位数、方差、标准差
- 概率分布：二项分布、正态分布、泊松分布
- 条件概率与贝叶斯定理
- 假设检验：t 检验、卡方检验
- 回归分析：线性回归、逻辑回归

贝叶斯定理

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

其中：

- $P(A|B)$：后验概率
- $P(B|A)$：似然度
- $P(A)$：先验概率
- $P(B)$：证据

线性代数

矩阵运算与应用

- 向量和矩阵运算：加法、乘法、转置

线性代数

矩阵运算与应用

- 向量和矩阵运算：加法、乘法、转置
- 特征值和特征向量

线性代数

矩阵运算与应用

- 向量和矩阵运算：加法、乘法、转置
- 特征值和特征向量
- 矩阵分解：SVD、PCA

奇异值分解（SVD

任何矩阵的通用分解方法

SVD 公式

对于任意 $m \times n$ 矩阵 A，其奇异值分解为：

$$A = U\Sigma V^\top$$

其中：

- U: $m \times m$ 正交矩阵（左奇异向量）
- Σ: $m \times n$ 对角矩阵（奇异值，按降序排列）
- $V^\top$: $n \times n$ 正交矩阵的转置（右奇异向量）

矩阵性质的详细说明

矩阵	数学性质	几何意义
U	▪ $U^\top U = I_m$ ▪ 列向量正交 ▪ 列向量单位长度	▪ 代表输出空间的旋转/反射 ▪ 列向量是 $AA^\top$ 的特征向量
Σ	▪ 对角矩阵 ▪ $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ ▪ $\sigma_{r+1} = \cdots = \sigma_{\min(m,n)} = 0$	▪ 奇异值表示缩放因子 ▪ 秩 $r =$ 非零奇异值个数 ▪ 条件数 $= \sigma_1/\sigma_r$
$V^\top$	▪ $V^\top V = I_n$ ▪ 行向量正交 ▪ V 的列是 $A^\top A$ 的特征向量	▪ 代表输入空间的旋转/反射 ▪ 行向量定义主成分方向

矩阵性质的详细说明

矩阵	数学性质	几何意义
U	▪ $U^\top U = I_m$ ▪ 列向量正交 ▪ 列向量单位长度	▪ 代表输出空间的旋转/反射 ▪ 列向量是 $AA^\top$ 的特征向量
Σ	▪ 对角矩阵 ▪ $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ ▪ $\sigma_{r+1} = \cdots = \sigma_{\min(m,n)} = 0$	▪ 奇异值表示缩放因子 ▪ 秩 $r =$ 非零奇异值个数 ▪ 条件数 $= \sigma_1/\sigma_r$
$V^\top$	▪ $V^\top V = I_n$ ▪ 行向量正交 ▪ V 的列是 $A^\top A$ 的特征向量	▪ 代表输入空间的旋转/反射 ▪ 行向量定义主成分方向

- 奇异值 σ_i 是非负实数，满足 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$

矩阵性质的详细说明

矩阵	数学性质	几何意义
U	▪ $U^\top U = I_m$ ▪ 列向量正交 ▪ 列向量单位长度	▪ 代表输出空间的旋转/反射 ▪ 列向量是 $AA^\top$ 的特征向量
Σ	▪ 对角矩阵 ▪ $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ ▪ $\sigma_{r+1} = \cdots = \sigma_{\min(m,n)} = 0$	▪ 奇异值表示缩放因子 ▪ 秩 $r =$ 非零奇异值个数 ▪ 条件数 $= \sigma_1/\sigma_r$
$V^\top$	▪ $V^\top V = I_n$ ▪ 行向量正交 ▪ V 的列是 $A^\top A$ 的特征向量	▪ 代表输入空间的旋转/反射 ▪ 行向量定义主成分方向

- 奇异值 σ_i 是非负实数，满足 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$
- r 是矩阵 A 的秩

矩阵性质的详细说明

矩阵	数学性质	几何意义
U	▪ $U^\top U = I_m$ ▪ 列向量正交 ▪ 列向量单位长度	▪ 代表输出空间的旋转/反射 ▪ 列向量是 $AA^\top$ 的特征向量
Σ	▪ 对角矩阵 ▪ $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ ▪ $\sigma_{r+1} = \cdots = \sigma_{\min(m,n)} = 0$	▪ 奇异值表示缩放因子 ▪ 秩 $r =$ 非零奇异值个数 ▪ 条件数 $= \sigma_1/\sigma_r$
$V^\top$	▪ $V^\top V = I_n$ ▪ 行向量正交 ▪ V 的列是 $A^\top A$ 的特征向量	▪ 代表输入空间的旋转/反射 ▪ 行向量定义主成分方向

- 奇异值 σ_i 是非负实数，满足 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$
- r 是矩阵 A 的秩
- U 和 V 都是正交矩阵，满足 $U^\top U = I$，$V^\top V = I$

矩阵性质的详细说明

矩阵	数学性质	几何意义
U	▪ $U^\top U = I_m$ ▪ 列向量正交 ▪ 列向量单位长度	▪ 代表输出空间的旋转/反射 ▪ 列向量是 $AA^\top$ 的特征向量
Σ	▪ 对角矩阵 ▪ $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ ▪ $\sigma_{r+1} = \cdots = \sigma_{\min(m,n)} = 0$	▪ 奇异值表示缩放因子 ▪ 秩 $r =$ 非零奇异值个数 ▪ 条件数 $= \sigma_1/\sigma_r$
$V^\top$	▪ $V^\top V = I_n$ ▪ 行向量正交 ▪ V 的列是 $A^\top A$ 的特征向量	▪ 代表输入空间的旋转/反射 ▪ 行向量定义主成分方向

- 奇异值 σ_i 是非负实数，满足 $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$
- r 是矩阵 A 的秩
- U 和 V 都是正交矩阵，满足 $U^\top U = I$，$V^\top V = I$
- 奇异值与特征值的关系：$\sigma_i = \sqrt{\lambda_i(A^\top A)}$

主成分分析与 SVD 分解

特性维度	PCA (主成分分析)	SVD (奇异值分解)
核心本质	一种**统计学**中的有明确目标的**降维方法**	一种**线性代数**中通用的**矩阵分解工具**
主要目标	寻找数据方差最大的方向，以保留最大“信息量”	将任意矩阵分解为三个特定矩阵的乘积，揭示其内在结构
输入要求	通常需要**中心化**（均值为 0）的数据矩阵	可以处理**任意**矩阵（包括非方阵、未中心化的数据）
数学输出	主成分（特征向量）和解释方差（特征值）	左奇异向量（U）、奇异值（Σ）、右奇异向量（V）
关键联系	**PCA 可以通过对中心化后的数据矩阵进行 SVD 来实现**	**SVD 的右奇异向量（V）就是 PCA 的主成分方向**

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解

- θ：模型参数

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解
- 凸优化：保证全局最优解

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解
- 凸优化：保证全局最优解
- 拉格朗日乘数法：约束优化

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解
- 凸优化：保证全局最优解
- 拉格朗日乘数法：约束优化
- 随机优化：处理大规模数据

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解
- 凸优化：保证全局最优解
- 拉格朗日乘数法：约束优化
- 随机优化：处理大规模数据

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

优化理论的应用

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解
- 凸优化：保证全局最优解
- 拉格朗日乘数法：约束优化
- 随机优化：处理大规模数据

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

优化理论的应用

- 机器学习模型训练

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解
- 凸优化：保证全局最优解
- 拉格朗日乘数法：约束优化
- 随机优化：处理大规模数据

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

优化理论的应用

- 机器学习模型训练
- 参数估计

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解
- 凸优化：保证全局最优解
- 拉格朗日乘数法：约束优化
- 随机优化：处理大规模数据

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

优化理论的应用

- 机器学习模型训练
- 参数估计
- 特征选择

微积分与优化理论

优化方法

- 梯度下降法：迭代逼近最优解
- 凸优化：保证全局最优解
- 拉格朗日乘数法：约束优化
- 随机优化：处理大规模数据

梯度下降

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

其中：

- θ：模型参数
- η：学习率
- $\nabla J(\theta)$：梯度

优化理论的应用

- 机器学习模型训练
- 参数估计
- 特征选择
- 超参数调优

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理
- 数据集分割
- 缺失值与异常值
- 特征选择与降维

4 传统机器学习模型

特征工程与预处理概述

重要性

特征工程是机器学习流程中的关键步骤，好的特征可以显著提升模型性能。数据预处理确保数据质量，为后续分析奠定基础。

特征工程主要流程

- 特征构造

特征工程与预处理概述

重要性

特征工程是机器学习流程中的关键步骤，好的特征可以显著提升模型性能。数据预处理确保数据质量，为后续分析奠定基础。

特征工程主要流程

- 特征构造，例如：
  - **领域知识驱动**：结合具体业务背景创造有意义的特征（如电商中从交易时间提取“是否周末”）。

特征工程与预处理概述

重要性

特征工程是机器学习流程中的关键步骤，好的特征可以显著提升模型性能。数据预处理确保数据质量，为后续分析奠定基础。

特征工程主要流程

- 特征构造，例如：
  - **领域知识驱动**：结合具体业务背景创造有意义的特征（如电商中从交易时间提取“是否周末”）。
  - **数学变换**：应用对数变换、多项式变换等改变数据分布或关系。

特征工程与预处理概述

重要性

特征工程是机器学习流程中的关键步骤，好的特征可以显著提升模型性能。数据预处理确保数据质量，为后续分析奠定基础。

特征工程主要流程

- 特征构造，例如：
  - **领域知识驱动**：结合具体业务背景创造有意义的特征（如电商中从交易时间提取“是否周末”）。
  - **数学变换**：应用对数变换、多项式变换等改变数据分布或关系。
  - **特征交互**：通过特征的加减乘除等运算捕捉变量间的协同效应。

特征工程与预处理概述

重要性

特征工程是机器学习流程中的关键步骤，好的特征可以显著提升模型性能。数据预处理确保数据质量，为后续分析奠定基础。

特征工程主要流程

- 特征构造，例如：
  - **领域知识驱动**：结合具体业务背景创造有意义的特征（如电商中从交易时间提取“是否周末”）。
  - **数学变换**：应用对数变换、多项式变换等改变数据分布或关系。
  - **特征交互**：通过特征的加减乘除等运算捕捉变量间的协同效应。
  - **聚合统计**：基于类别或 ID 对数值特征进行分组统计（如求均值、最大值等）。

特征工程与预处理概述

重要性

特征工程是机器学习流程中的关键步骤，好的特征可以显著提升模型性能。数据预处理确保数据质量，为后续分析奠定基础。

特征工程主要流程

- 特征构造
- 特征缩放与归一化：统一数据尺度

特征工程与预处理概述

重要性

特征工程是机器学习流程中的关键步骤，好的特征可以显著提升模型性能。数据预处理确保数据质量，为后续分析奠定基础。

特征工程主要流程

- 特征构造
- 特征缩放与归一化：统一数据尺度
- 处理缺失值与异常值：保证数据完整性

特征工程与预处理概述

重要性

特征工程是机器学习流程中的关键步骤，好的特征可以显著提升模型性能。数据预处理确保数据质量，为后续分析奠定基础。

特征工程主要流程

- 特征构造
- 特征缩放与归一化：统一数据尺度
- 处理缺失值与异常值：保证数据完整性
- 特征选择与降维：提高模型效率和性能

特征缩放与归一化

标准化 (Z-score)

$$z = \frac{x - \mu}{\sigma}$$

- μ：特征均值
- σ：特征标准差
- 结果均值为 0，标准差为 1

特征缩放与归一化

标准化 (Z-score)

$$z = \frac{x - \mu}{\sigma}$$

- μ：特征均值
- σ：特征标准差
- 结果均值为 0，标准差为 1

最小-最大缩放

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

将特征缩放到 [0, 1] 区间

特征缩放与归一化

Robust 缩放（鲁棒缩放）

- **原理**：使用中位数和四分位距（IQR）进行缩放，对异常值不敏感
- **公式**：

$$X_{\text{scaled}} = \frac{X - \text{Median}(X)}{Q_3 - Q_1}$$

其中 Q_1、Q_3 为第一、第三四分位数

- **优点**：抗异常值能力强，保持数据分布稳定性

特征缩放与归一化

Robust 缩放（鲁棒缩放）

- **原理**：使用中位数和四分位距（IQR）进行缩放，对异常值不敏感
- **公式**：

$$X_{\text{scaled}} = \frac{X - \text{Median}(X)}{Q_3 - Q_1}$$

其中 Q_1、Q_3 为第一、第三四分位数

- **优点**：抗异常值能力强，保持数据分布稳定性

L2 缩放（单位范数归一化）

- **原理**：将每个样本缩放到 L2 范数（欧几里得范数）为 1
- **公式**：

$$x_{\text{scaled}} = \frac{x}{\|x\|_2} = \frac{x}{\sqrt{\sum x_i^2}}$$

- **优点**：保持向量方向，适用于距离度量
- **适用场景**：文本分类、聚类算法、余弦相似度计算

归一化方法比较

方法	适用范围	优点
标准化	分布近似高斯	保持异常值信息
最小-最大	边界已知	保持原始分布
Robust 缩放	有异常值	减少异常值影响
L2 归一化	向量数据	保持方向信息

注意事项

- 必须在训练集上计算参数，然后应用到测试集
- 避免数据泄露问题
- 选择适合数据特性的缩放方法

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理
- 数据集分割
  - 主要分割方法详解
  - 实践建议与总结
- 缺失值与异常值
- 特征选择与降维

4 传统机器学习模型

数据集分割的基本概念

为什么需要分割数据集?

数据集分割是机器学习流程中的关键步骤，旨在评估模型在未见数据上的泛化能力，防止过拟合，并提供客观的性能评估。

数据集分割的基本概念

为什么需要分割数据集?

数据集分割是机器学习流程中的关键步骤，旨在评估模型在未见数据上的泛化能力，防止过拟合，并提供客观的性能评估。

训练集 (Training Set)

- 用于模型参数学习
- 占数据集大部分比例
- 模型通过训练集学习数据规律

数据集分割的基本概念

为什么需要分割数据集?

数据集分割是机器学习流程中的关键步骤，旨在评估模型在未见数据上的泛化能力，防止过拟合，并提供客观的性能评估。

训练集 (Training Set)

- 用于模型参数学习
- 占数据集大部分比例
- 模型通过训练集学习数据规律

验证集 (Validation Set)

- 用于超参数调优和模型选择
- 防止过拟合
- 中间评估与调整

数据集分割的基本概念

为什么需要分割数据集?

数据集分割是机器学习流程中的关键步骤，旨在评估模型在未见数据上的泛化能力，防止过拟合，并提供客观的性能评估。

训练集 (Training Set)

- 用于模型参数学习
- 占数据集大部分比例
- 模型通过训练集学习数据规律

验证集 (Validation Set)

- 用于超参数调优和模型选择
- 防止过拟合
- 中间评估与调整

测试集 (Test Set)

- 最终性能客观评估
- 模拟真实场景的未见数据
- 不参与任何训练过程

数据集分割的基本概念

为什么需要分割数据集?

数据集分割是机器学习流程中的关键步骤，旨在评估模型在未见数据上的泛化能力，防止过拟合，并提供客观的性能评估。

数据分割的核心原则

测试集必须与训练集/验证集互斥，且在整个模型开发周期中仅使用一次，用于最终评估。

数据集分割方法概览

数据集分割方法比较

方法	数据利用率	稳定性	适用场景
留出法 (Hold-out)	低	低	大数据集快速验证
K 折交叉验证 (K-Fold CV)	高	中高	中小数据集稳定评估
分层交叉验证 (Stratified CV)	高	高	类别不平衡数据
留一法 (Leave-One-Out)	最高	最高	极小数据集无偏估计
自助法 (Bootstrap)	高	中	小数据集，集成学习
时间序列分割	中	中	时间序列数据

数据集分割方法选择准则

- **数据量大小**：小数据用交叉验证或自助法，大数据用留出法

数据集分割方法选择准则

- **数据量大小**：小数据用交叉验证或自助法，大数据用留出法
- **数据特性**：时序数据用时间序列分割，类别不平衡用分层抽样

数据集分割方法选择准则

- **数据量大小**：小数据用交叉验证或自助法，大数据用留出法
- **数据特性**：时序数据用时间序列分割，类别不平衡用分层抽样
- **计算资源**：资源有限时优先选择留出法

数据集分割方法选择准则

- **数据量大小**：小数据用交叉验证或自助法，大数据用留出法
- **数据特性**：时序数据用时间序列分割，类别不平衡用分层抽样
- **计算资源**：资源有限时优先选择留出法
- **评估稳定性**：重要项目建议使用交叉验证

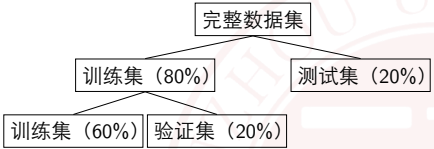
1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理
- 数据集分割
  - 主要分割方法详解
  - 实践建议与总结
- 缺失值与异常值
- 特征选择与降维

4 传统机器学习模型

留出法 (Hold-out Method)

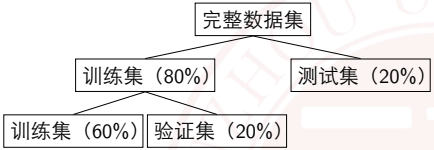


留出法分割示意图

方法原理

将数据集随机划分为两个互斥部分：训练集和测试集。可实现训练/验证/测试三部分划分。

留出法 (Hold-out Method)



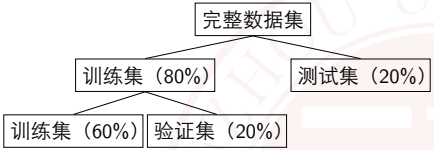
留出法分割示意图

方法原理

将数据集随机划分为两个互斥部分：训练集和测试集。可实现训练/验证/测试三部分划分。

- **优点**：实现简单，计算效率高

留出法 (Hold-out Method)



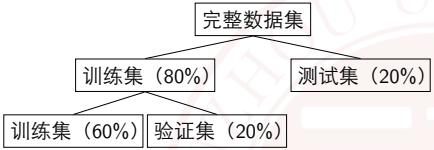
留出法分割示意图

方法原理

将数据集随机划分为两个互斥部分：训练集和测试集。可实现训练/验证/测试三部分划分。

- **优点**：实现简单，计算效率高
- **缺点**：结果受随机划分影响大，数据利用不充分

留出法 (Hold-out Method)



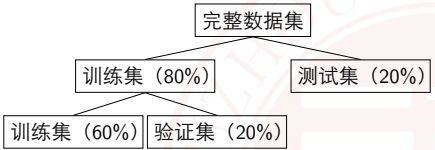
留出法分割示意图

方法原理

将数据集随机划分为两个互斥部分：训练集和测试集。可实现训练/验证/测试三部分划分。

- **优点**：实现简单，计算效率高
- **缺点**：结果受随机划分影响大，数据利用不充分
- **适用场景**：大数据集快速验证

留出法 (Hold-out Method)

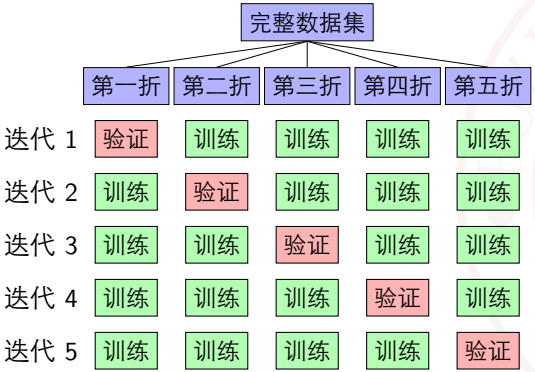


留出法分割示意图

典型分割比例

- 小规模数据：60% 训练，20% 验证，20% 测试
- 中等规模：70-80% 训练，10-15% 验证，10-15% 测试
- 大规模数据：98% 训练，1% 验证，1% 测试

K 折交叉验证 (K-Fold Cross-Validation)

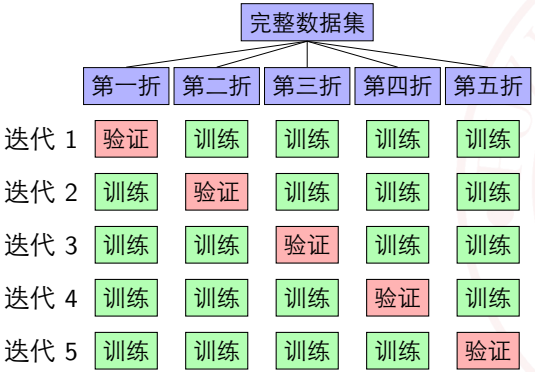


K 折交叉验证示意图（K=5）

方法原理

将数据集分为 K 个大小相等的互斥子集，每次用 K-1 个子集训练，剩余 1 个子集验证，重复 K 次，取平均结果。

K 折交叉验证 (K-Fold Cross-Validation)



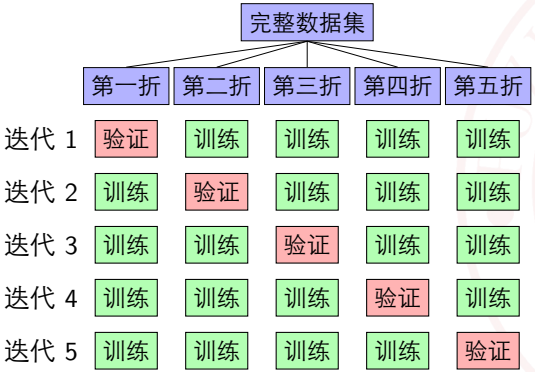
K 折交叉验证示意图（K=5）

方法原理

将数据集分为 K 个大小相等的互斥子集，每次用 K-1 个子集训练，剩余 1 个子集验证，重复 K 次，取平均结果。

- **优点**：数据利用充分，评估结果稳定

K 折交叉验证 (K-Fold Cross-Validation)



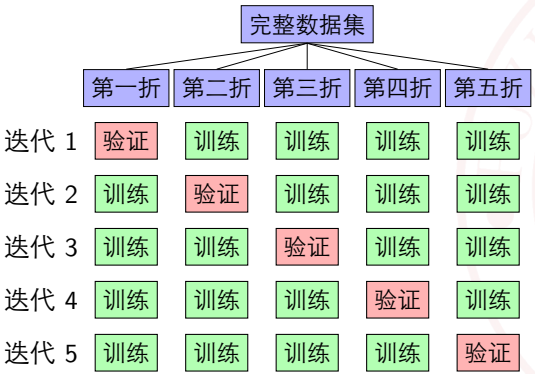
K 折交叉验证示意图（K=5）

方法原理

将数据集分为 K 个大小相等的互斥子集，每次用 K-1 个子集训练，剩余 1 个子集验证，重复 K 次，取平均结果。

- **优点**：数据利用充分，评估结果稳定
- **缺点**：计算成本高，不适合超大数据集

K 折交叉验证 (K-Fold Cross-Validation)



K 折交叉验证示意图（K=5）

方法原理

将数据集分为 K 个大小相等的互斥子集，每次用 K-1 个子集训练，剩余 1 个子集验证，重复 K 次，取平均结果。

- **优点**：数据利用充分，评估结果稳定
- **缺点**：计算成本高，不适合超大数据集
- **K 值选择**：常用 5 或 10，平衡计算成本与稳定性

K 折交叉验证 (K-Fold Cross-Validation)

方法原理

将数据集分为 K 个大小相等的互斥子集，每次用 K-1 个子集训练，剩余 1 个子集验证，重复 K 次，取平均结果。

- **优点**：数据利用充分，评估结果稳定
- **缺点**：计算成本高，不适合超大数据集
- **K 值选择**：常用 5 或 10，平衡计算成本与稳定性

注意事项

- 划分前应打乱数据顺序

K 折交叉验证 (K-Fold Cross-Validation)

方法原理

将数据集分为 K 个大小相等的互斥子集，每次用 K-1 个子集训练，剩余 1 个子集验证，重复 K 次，取平均结果。

- **优点**：数据利用充分，评估结果稳定
- **缺点**：计算成本高，不适合超大数据集
- **K 值选择**：常用 5 或 10，平衡计算成本与稳定性

注意事项

- 划分前应打乱数据顺序
- 对于分类问题，推荐使用分层 K 折交叉验证

K 折交叉验证 (K-Fold Cross-Validation)

方法原理

将数据集分为 K 个大小相等的互斥子集，每次用 K-1 个子集训练，剩余 1 个子集验证，重复 K 次，取平均结果。

- **优点**：数据利用充分，评估结果稳定
- **缺点**：计算成本高，不适合超大数据集
- **K 值选择**：常用 5 或 10，平衡计算成本与稳定性

注意事项

- 划分前应打乱数据顺序
- 对于分类问题，推荐使用分层 K 折交叉验证
- 可重复多次 K 折验证取平均值提高稳定性

分层交叉验证与留一法

分层交叉验证 (Stratified CV)

保持每个折中类别比例与原始数据集一致，特别适用于类别不平衡数据。

分层交叉验证与留一法

分层交叉验证 (Stratified CV)

保持每个折中类别比例与原始数据集一致，特别适用于类别不平衡数据。

留一法 (Leave-One-Out, LOO)

K 折交叉验证的特殊形式，K 等于样本数 N。每个样本单独作为测试集，其余作为训练集。

分层交叉验证与留一法

分层交叉验证 (Stratified CV)

保持每个折中类别比例与原始数据集一致，特别适用于类别不平衡数据。

留一法 (Leave-One-Out, LOO)

K 折交叉验证的特殊形式，K 等于样本数 N。每个样本单独作为测试集，其余作为训练集。

- **优点**：无偏估计，数据利用最充分

分层交叉验证与留一法

分层交叉验证 (Stratified CV)

保持每个折中类别比例与原始数据集一致，特别适用于类别不平衡数据。

留一法 (Leave-One-Out, LOO)

K 折交叉验证的特殊形式，K 等于样本数 N。每个样本单独作为测试集，其余作为训练集。

- **优点**：无偏估计，数据利用最充分
- **缺点**：计算成本极高，仅适合小数据集

分层交叉验证与留一法

分层交叉验证 (Stratified CV)

保持每个折中类别比例与原始数据集一致，特别适用于类别不平衡数据。

留一法 (Leave-One-Out, LOO)

K 折交叉验证的特殊形式，K 等于样本数 N。每个样本单独作为测试集，其余作为训练集。

- **优点**：无偏估计，数据利用最充分
- **缺点**：计算成本极高，仅适合小数据集

留一法数学表示

对于包含 N 个样本的数据集，留一法的误差估计为：

$$\text{误差} = \frac{1}{N}\sum_{i=1}^{N} L(y_i, f_{(-i)}(x_i))$$

其中 $f_{(-i)}$ 表示使用除第 i 个样本外所有样本训练的模型。

自助法 (Bootstrap Method)

方法原理

有放回随机抽样生成训练集，未被抽中的样本作为测试集（袋外数据，OOB）。适用于小数据集和置信区间估计。

- 每个样本未被抽中的概率约为 36.8%
- 可生成多个自助样本进行统计估计

自助法数学原理

从包含 N 个样本的数据集中有放回地抽取 N 次，一个样本在整个 N 次抽取中始终不被抽中的概率为：

$$\lim_{N\to\infty}\left(1-\frac{1}{N}\right)^{N}=\frac{1}{e}\approx 0.368$$

优缺点分析

- **优点**：适用于小数据集，可用于估计置信区间
- **缺点**：改变原始数据分布，引入估计偏差

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理
- 数据集分割
  - 主要分割方法详解
  - 实践建议与总结
- 缺失值与异常值
- 特征选择与降维

4 传统机器学习模型

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

常见陷阱与避免方法

- **数据泄露 (Data Leakage)**：测试集信息无意中进入训练过程

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

常见陷阱与避免方法

- **数据泄露 (Data Leakage)**：测试集信息无意中进入训练过程
- **抽样偏差**：划分后的数据集不能代表总体分布

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

常见陷阱与避免方法

- **数据泄露 (Data Leakage)**：测试集信息无意中进入训练过程
- **抽样偏差**：划分后的数据集不能代表总体分布
- **测试集过使用**：多次基于测试集调整模型

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

实用工作流程

1. 探索性数据分析

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

实用工作流程

1. 探索性数据分析 $\rightarrow$ 2. 分层划分数据集

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

实用工作流程

1. 探索性数据分析 $\rightarrow$ 2. 分层划分数据集 $\rightarrow$ 3. 在训练集上训练模型

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

实用工作流程

1. 探索性数据分析 $\rightarrow$ 2. 分层划分数据集 $\rightarrow$ 3. 在训练集上训练模型 $\rightarrow$ 4. 在验证集上调参

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

实用工作流程

1. 探索性数据分析 $\rightarrow$ 2. 分层划分数据集 $\rightarrow$ 3. 在训练集上训练模型 $\rightarrow$ 4. 在验证集上调参 $\rightarrow$ 5. 最终在测试集上评估

最佳实践与常见陷阱

最佳实践建议

- **分层抽样**：保持训练/验证/测试集的类别分布一致
- **随机种子固定**：确保结果可重现
- **时序数据顺序**：严格按时间顺序划分，防止数据泄露
- **多次验证**：重要项目进行多次随机划分取平均结果

实用工作流程

1. 探索性数据分析 $\rightarrow$ 2. 分层划分数据集 $\rightarrow$ 3. 在训练集上训练模型 $\rightarrow$ 4. 在验证集上调参 $\rightarrow$ 5. 最终在测试集上评估 $\rightarrow$ 6. 结果分析与报告

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理
- 数据集分割
- 缺失值与异常值
- 特征选择与降维

4 传统机器学习模型

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征

缺失值模式

使用统计方法检测缺失模式：

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征
- 填充：

缺失值模式

使用统计方法检测缺失模式：

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征
- 填充：
  - 均值/中位数/众数填充

缺失值模式

使用统计方法检测缺失模式：

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征
- 填充：
  - 均值/中位数/众数填充
  - 插值法：线性、多项式

缺失值模式

使用统计方法检测缺失模式：

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征
- 填充：
  - 均值/中位数/众数填充
  - 插值法：线性、多项式
- 模型预测填充

缺失值模式

使用统计方法检测缺失模式：

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征
- 填充：
  - 均值/中位数/众数填充
  - 插值法：线性、多项式
  - 模型预测填充
  - 特定值填充（如-999）

缺失值模式

使用统计方法检测缺失模式：

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征
- 填充：
  - 均值/中位数/众数填充
  - 插值法：线性、多项式
  - 模型预测填充
  - 特定值填充（如-999）
- 作为单独类别处理

缺失值模式

使用统计方法检测缺失模式：

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征
- 填充：
  - 均值/中位数/众数填充
  - 插值法：线性、多项式
  - 模型预测填充
  - 特定值填充（如-999）
- 作为单独类别处理

缺失值模式

使用统计方法检测缺失模式：

- MCAR：完全随机缺失

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征
- 填充：
  - 均值/中位数/众数填充
  - 插值法：线性、多项式
  - 模型预测填充
  - 特定值填充（如-999）
- 作为单独类别处理

缺失值模式

使用统计方法检测缺失模式：

- MCAR：完全随机缺失
- MAR：随机缺失

处理缺失值

缺失值处理策略

- 删除：缺失比例高的样本或特征
- 填充：
  - 均值/中位数/众数填充
  - 插值法：线性、多项式
  - 模型预测填充
  - 特定值填充（如-999）
- 作为单独类别处理

缺失值模式

使用统计方法检测缺失模式：

- MCAR：完全随机缺失
- MAR：随机缺失
- MNAR：非随机缺失

处理异常值

异常值检测方法

- 统计方法:

异常值处理

处理异常值

异常值检测方法

- 统计方法：
  - Z-score（$|z| > 3$）——偏差超过 3 个标准差

异常值处理

处理异常值

异常值检测方法

- 统计方法：
  - Z-score（$|z| > 3$）——偏差超过 3 个标准差
  - IQR 方法（Q1-1.5IQR, Q3+1.5IQR）

异常值处理

处理异常值

异常值检测方法

- 统计方法：
  - Z-score（$|z| > 3$）——偏差超过 3 个标准差
  - IQR 方法（Q1-1.5IQR, Q3+1.5IQR）
- 可视化方法：箱线图、散点图

异常值处理

处理异常值

异常值检测方法

- 统计方法：
  - Z-score（$|z| > 3$）——偏差超过 3 个标准差
  - IQR 方法（Q1-1.5IQR, Q3+1.5IQR）
- 可视化方法：箱线图、散点图
- 模型方法：隔离森林、DBSCAN

异常值处理

处理异常值

异常值检测方法

- 统计方法：
  - Z-score（$|z| > 3$）——偏差超过 3 个标准差
  - IQR 方法（Q1-1.5IQR, Q3+1.5IQR）
- 可视化方法：箱线图、散点图
- 模型方法：隔离森林、DBSCAN
- 距离方法：基于聚类的异常检测

异常值处理

处理异常值

异常值检测方法

- 统计方法：
  - Z-score（$|z| > 3$）——偏差超过 3 个标准差
  - IQR 方法（Q1-1.5IQR, Q3+1.5IQR）
- 可视化方法：箱线图、散点图
- 模型方法：隔离森林、DBSCAN
- 距离方法：基于聚类的异常检测

异常值处理

- 删除：确认是错误记录

处理异常值

异常值检测方法

- 统计方法：
  - Z-score（$|z| > 3$）——偏差超过 3 个标准差
  - IQR 方法（Q1-1.5IQR, Q3+1.5IQR）
- 可视化方法：箱线图、散点图
- 模型方法：隔离森林、DBSCAN
- 距离方法：基于聚类的异常检测

异常值处理

- 删除：确认是错误记录
- 修正：纠正明显错误

处理异常值

异常值检测方法

- 统计方法：
  - Z-score（$|z| > 3$）——偏差超过 3 个标准差
  - IQR 方法（Q1-1.5IQR, Q3+1.5IQR）
- 可视化方法：箱线图、散点图
- 模型方法：隔离森林、DBSCAN
- 距离方法：基于聚类的异常检测

异常值处理

- 删除：确认是错误记录
- 修正：纠正明显错误
- 替换：用边界值替换

处理异常值

异常值检测方法

- 统计方法：
  - Z-score（$|z| > 3$）——偏差超过 3 个标准差
  - IQR 方法（Q1-1.5IQR, Q3+1.5IQR）
- 可视化方法：箱线图、散点图
- 模型方法：隔离森林、DBSCAN
- 距离方法：基于聚类的异常检测

异常值处理

- 删除：确认是错误记录
- 修正：纠正明显错误
- 替换：用边界值替换
- 保留：如果是真实现象

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理
- 数据集分割
- 缺失值与异常值
- 特征选择与降维

4 传统机器学习模型

特征选择与降维的重要性

- 处理高维数据时面临维度灾难问题

特征选择与降维的重要性

- 处理高维数据时面临维度灾难问题
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。

特征选择与降维的重要性

- **处理高维数据时面临维度灾难问题**
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。

特征选择与降维的重要性

- 处理高维数据时面临维度灾难问题
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。
- 减少计算成本，提高训练效率

特征选择与降维的重要性

- **处理高维数据时面临维度灾难问题**
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。
- **减少计算成本，提高训练效率**
  - 减少特征数量直接降低模型复杂度，缩短训练和推理时间。

特征选择与降维的重要性

- **处理高维数据时面临维度灾难问题**
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。
- **减少计算成本，提高训练效率**
  - 减少特征数量直接降低模型复杂度，缩短训练和推理时间。
  - 更少的参数需要优化，计算资源需求显著下降，迭代速度加快。

特征选择与降维的重要性

- **处理高维数据时面临维度灾难问题**
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。
- **减少计算成本，提高训练效率**
  - 减少特征数量直接降低模型复杂度，缩短训练和推理时间。
  - 更少的参数需要优化，计算资源需求显著下降，迭代速度加快。
- **提升模型泛化能力，避免过拟合**

特征选择与降维的重要性

- **处理高维数据时面临维度灾难问题**
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。
- **减少计算成本，提高训练效率**
  - 减少特征数量直接降低模型复杂度，缩短训练和推理时间。
  - 更少的参数需要优化，计算资源需求显著下降，迭代速度加快。
- **提升模型泛化能力，避免过拟合**
  - 剔除不相关和冗余特征，减少模型对训练数据噪声的学习。

特征选择与降维的重要性

- **处理高维数据时面临维度灾难问题**
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。
- **减少计算成本，提高训练效率**
  - 减少特征数量直接降低模型复杂度，缩短训练和推理时间。
  - 更少的参数需要优化，计算资源需求显著下降，迭代速度加快。
- **提升模型泛化能力，避免过拟合**
  - 剔除不相关和冗余特征，减少模型对训练数据噪声的学习。
  - 简化模型结构，增强对数据本质规律的捕捉能力，提高在新数据上的表现。

特征选择与降维的重要性

- **处理高维数据时面临维度灾难问题**
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。
- **减少计算成本，提高训练效率**
  - 减少特征数量直接降低模型复杂度，缩短训练和推理时间。
  - 更少的参数需要优化，计算资源需求显著下降，迭代速度加快。
- **提升模型泛化能力，避免过拟合**
  - 剔除不相关和冗余特征，减少模型对训练数据噪声的学习。
  - 简化模型结构，增强对数据本质规律的捕捉能力，提高在新数据上的表现。
- **增强模型可解释性和可视化能力**

特征选择与降维的重要性

- **处理高维数据时面临维度灾难问题**
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。
- **减少计算成本，提高训练效率**
  - 减少特征数量直接降低模型复杂度，缩短训练和推理时间。
  - 更少的参数需要优化，计算资源需求显著下降，迭代速度加快。
- **提升模型泛化能力，避免过拟合**
  - 剔除不相关和冗余特征，减少模型对训练数据噪声的学习。
  - 简化模型结构，增强对数据本质规律的捕捉能力，提高在新数据上的表现。
- **增强模型可解释性和可视化能力**
  - 低维特征空间更符合人类认知，便于理解特征与目标之间的关系。

特征选择与降维的重要性

- **处理高维数据时面临维度灾难问题**
  - 高维空间中数据稀疏性增加，样本间距离计算失效，模型难以学习有效规律。
  - 特征数量增加导致计算复杂度呈指数级增长，存储和计算资源消耗巨大。
- **减少计算成本，提高训练效率**
  - 减少特征数量直接降低模型复杂度，缩短训练和推理时间。
  - 更少的参数需要优化，计算资源需求显著下降，迭代速度加快。
- **提升模型泛化能力，避免过拟合**
  - 剔除不相关和冗余特征，减少模型对训练数据噪声的学习。
  - 简化模型结构，增强对数据本质规律的捕捉能力，提高在新数据上的表现。
- **增强模型可解释性和可视化能力**
  - 低维特征空间更符合人类认知，便于理解特征与目标之间的关系。
  - 降维至二维或三维后可实现数据分布可视化，辅助模式识别和异常检测。

特征选择三大方法

- 过滤法（Filter Methods）

特征选择三大方法

- **过滤法（Filter Methods）**
  - 基于统计指标评估特征与目标变量的相关性，独立于后续学习算法。

特征选择三大方法

- **过滤法（Filter Methods）**
  - 基于统计指标评估特征与目标变量的相关性，独立于后续学习算法。
  - 计算效率高，适合高维数据的初步筛选，但可能忽略特征间的交互作用。

特征选择三大方法

- **过滤法（Filter Methods）**
  - 基于统计指标评估特征与目标变量的相关性，独立于后续学习算法。
  - 计算效率高，适合高维数据的初步筛选，但可能忽略特征间的交互作用。
- **包裹法（Wrapper Methods）**

特征选择三大方法

- **过滤法（Filter Methods）**
  - 基于统计指标评估特征与目标变量的相关性，独立于后续学习算法。
  - 计算效率高，适合高维数据的初步筛选，但可能忽略特征间的交互作用。
- **包裹法（Wrapper Methods）**
  - 通过特定模型的性能表现来评估特征子集的质量，如递归特征消除。

特征选择三大方法

- **过滤法（Filter Methods）**
  - 基于统计指标评估特征与目标变量的相关性，独立于后续学习算法。
  - 计算效率高，适合高维数据的初步筛选，但可能忽略特征间的交互作用。
- **包裹法（Wrapper Methods）**
  - 通过特定模型的性能表现来评估特征子集的质量，如递归特征消除。
  - 考虑特征组合效应，通常能获得性能更优的特征子集，但计算成本较高。

特征选择三大方法

- **过滤法（Filter Methods）**
  - 基于统计指标评估特征与目标变量的相关性，独立于后续学习算法。
  - 计算效率高，适合高维数据的初步筛选，但可能忽略特征间的交互作用。
- **包裹法（Wrapper Methods）**
  - 通过特定模型的性能表现来评估特征子集的质量，如递归特征消除。
  - 考虑特征组合效应，通常能获得性能更优的特征子集，但计算成本较高。
- **嵌入法（Embedded Methods）**

特征选择三大方法

- 过滤法（Filter Methods）
  - 基于统计指标评估特征与目标变量的相关性，独立于后续学习算法。
  - 计算效率高，适合高维数据的初步筛选，但可能忽略特征间的交互作用。
- 包裹法（Wrapper Methods）
  - 通过特定模型的性能表现来评估特征子集的质量，如递归特征消除。
  - 考虑特征组合效应，通常能获得性能更优的特征子集，但计算成本较高。
- 嵌入法（Embedded Methods）
  - 将特征选择过程融入模型训练中，如 L1 正则化、决策树的特征重要性。

特征选择三大方法

- **过滤法（Filter Methods）**
  - 基于统计指标评估特征与目标变量的相关性，独立于后续学习算法。
  - 计算效率高，适合高维数据的初步筛选，但可能忽略特征间的交互作用。
- **包裹法（Wrapper Methods）**
  - 通过特定模型的性能表现来评估特征子集的质量，如递归特征消除。
  - 考虑特征组合效应，通常能获得性能更优的特征子集，但计算成本较高。
- **嵌入法（Embedded Methods）**
  - 将特征选择过程融入模型训练中，如 L1 正则化、决策树的特征重要性。
  - 平衡效率与效果，在训练过程中自动完成特征选择，适合大规模数据。

降维方法：PCA 与 LDA

- **PCA（主成分分析）**

降维方法：PCA 与 LDA

- **PCA（主成分分析）**
  - 无监督线性降维方法，通过正交变换将数据投影到方差最大的方向上。

降维方法：PCA 与 LDA

- **PCA（主成分分析）**
  - 无监督线性降维方法，通过正交变换将数据投影到方差最大的方向上。
  - 旨在保留数据最大变异信息，适用于数据压缩、去噪和可视化。

降维方法：PCA 与 LDA

- **PCA（主成分分析）**
  - 无监督线性降维方法，通过正交变换将数据投影到方差最大的方向上。
  - 旨在保留数据最大变异信息，适用于数据压缩、去噪和可视化。
- **LDA（线性判别分析）**

降维方法：PCA 与 LDA

- **PCA（主成分分析）**
  - 无监督线性降维方法，通过正交变换将数据投影到方差最大的方向上。
  - 旨在保留数据最大变异信息，适用于数据压缩、去噪和可视化。
- **LDA（线性判别分析）**
  - 有监督降维方法，最大化类间距离同时最小化类内距离。

降维方法：PCA 与 LDA

- **PCA（主成分分析）**
  - 无监督线性降维方法，通过正交变换将数据投影到方差最大的方向上。
  - 旨在保留数据最大变异信息，适用于数据压缩、去噪和可视化。
- **LDA（线性判别分析）**
  - 有监督降维方法，最大化类间距离同时最小化类内距离。
  - 利用类别标签信息，追求最佳类别分离效果，特别适用于分类任务。

降维方法：PCA 与 LDA

- **PCA（主成分分析）**
  - 无监督线性降维方法，通过正交变换将数据投影到方差最大的方向上。
  - 旨在保留数据最大变异信息，适用于数据压缩、去噪和可视化。
- **LDA（线性判别分析）**
  - 有监督降维方法，最大化类间距离同时最小化类内距离。
  - 利用类别标签信息，追求最佳类别分离效果，特别适用于分类任务。

核心区别与选择依据

PCA 不考虑类别信息，适用于无监督场景和探索性分析；LDA 利用类别标签，更适合有监督分类任务。选择取决于问题是否有标签以及最终目标。

方法选择与实践建议

- **数据规模与问题类型导向**
  - 高维小样本数据可优先考虑过滤法或 PCA 进行初步降维。
  - 计算资源充足且追求最优性能时，可尝试包裹法或嵌入式方法。
- **结合业务理解与模型反馈**
  - 领域知识有助于判断特征合理性，避免剔除有实际意义的特征。
  - 通过交叉验证评估不同特征子集或降维结果对模型性能的影响。
- **流程化与迭代优化**
  - 建立从数据探索到效果评估的完整流程，确保每一步可追溯。
  - 根据模型表现和业务需求反复调整特征选择或降维策略。

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
线性模型
非线性模型
时间序列模型

传统机器学习模型概览

学习任务	线性模型	非线性模型
有监督-回归	线性回归 (Linear Regression) 岭回归 (Ridge Regression) 套索回归 (Lasso Regression) 支持向量回归 (SVR)	决策树回归 随机森林回归 梯度提升回归
有监督-分类	逻辑回归 (Logistic Regression) 线性判别分析 (LDA) 朴素贝叶斯 (Naive Bayes) 支持向量机 (SVM)	决策树 (Decision Tree) K 近邻 (KNN) 随机森林 (Random Forest) 梯度提升树 (GBDT/XGBoost)
无监督	主成分分析 (PCA) 因子分析 (Factor Analysis) 独立成分分析 (ICA)	K 均值聚类 (K-Means) 层次聚类 DBSCAN 高斯混合模型 (GMM) t-SNE

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
  - 线性回归
  - 岭回归与 LASSO 回归
  - 逻辑回归
  - 支持向量机
- 非线性模型
- 时间序列模型

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
  - 线性回归
  - 岭回归与 LASSO 回归
  - 逻辑回归
  - 支持向量机
- 非线性模型
- 时间序列模型

基本概念

- **定义**：通过线性关系描述因变量与自变量之间依赖关系的统计方法

基本概念

- **定义**：通过线性关系描述因变量与自变量之间依赖关系的统计方法
- **模型形式**:

基本概念

- **定义**：通过线性关系描述因变量与自变量之间依赖关系的统计方法
- **模型形式**:
  - 一元线性回归：$y = \beta_0 + \beta_1 x + \varepsilon$

基本概念

- **定义**：通过线性关系描述因变量与自变量之间依赖关系的统计方法
- **模型形式**:
  - 一元线性回归：$y = \beta_0 + \beta_1 x + \varepsilon$
  - 多元线性回归：$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$

基本概念

- **定义**：通过线性关系描述因变量与自变量之间依赖关系的统计方法
- **模型形式**:
  - 一元线性回归：$y = \beta_0 + \beta_1 x + \varepsilon$
  - 多元线性回归：$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$
- **参数含义**:

基本概念

- **定义**：通过线性关系描述因变量与自变量之间依赖关系的统计方法
- **模型形式**:
  - 一元线性回归：$y = \beta_0 + \beta_1 x + \varepsilon$
  - 多元线性回归：$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$
- **参数含义**:
  - β_0：截距项，表示自变量全为 0 时的基准值

基本概念

- **定义**：通过线性关系描述因变量与自变量之间依赖关系的统计方法
- **模型形式**:
  - 一元线性回归：$y = \beta_0 + \beta_1 x + \varepsilon$
  - 多元线性回归：$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$
- **参数含义**:
  - β_0：截距项，表示自变量全为 0 时的基准值
  - β_j：回归系数，表示自变量 x_j 对 y 的影响强度

基本概念

- **定义**：通过线性关系描述因变量与自变量之间依赖关系的统计方法
- **模型形式**:
  - 一元线性回归：$y = \beta_0 + \beta_1 x + \varepsilon$
  - 多元线性回归：$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$
- **参数含义**:
  - β_0：截距项，表示自变量全为 0 时的基准值
  - β_j：回归系数，表示自变量 x_j 对 y 的影响强度
  - ε：误差项，服从均值为 0 的正态分布

损失函数与最小二乘法

- 目标：找到一组参数使得预测值与真实值的误差最小

损失函数与最小二乘法

- **目标**：找到一组参数使得预测值与真实值的误差最小
- **损失函数**：残差平方和（RSS）

$$\text{RSS} = \sum_{i=1}^{m}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{m}(y_i - (\beta_0 + \beta_1 x_i))^2$$

损失函数与最小二乘法

- **目标**：找到一组参数使得预测值与真实值的误差最小
- **损失函数**：残差平方和（RSS）

$$\text{RSS} = \sum_{i=1}^{m}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{m}(y_i - (\beta_0 + \beta_1 x_i))^2$$

- **最小二乘法原理**：通过最小化 RSS 来估计参数

$$\min_{\beta_0,\beta_1} \sum_{i=1}^{m}(y_i - \beta_0 - \beta_1 x_i)^2$$

损失函数与最小二乘法

- **目标**：找到一组参数使得预测值与真实值的误差最小
- **损失函数**：残差平方和（RSS）

$$\text{RSS} = \sum_{i=1}^{m}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{m}(y_i - (\beta_0 + \beta_1 x_i))^2$$

- **最小二乘法原理**：通过最小化 RSS 来估计参数

$$\min_{\beta_0,\beta_1} \sum_{i=1}^{m}(y_i - \beta_0 - \beta_1 x_i)^2$$

- **直观理解**：类似“拉皮筋”让直线尽量贴近所有数据点

一元线性回归推导

- **正规方程**：对 β_0 和 β_1 分别求偏导并令其为 0

$$\frac{\partial \text{RSS}}{\partial \beta_0} = -2\sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i) = 0$$

$$\frac{\partial \text{RSS}}{\partial \beta_1} = -2\sum_{i=1}^{n} x_i(y_i - \beta_0 - \beta_1 x_i) = 0$$

- **推导结果**:

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

$$\beta_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

- **几何意义**：斜率表示 x 与 y 的协方差与 x 方差的比值

矩阵形式推导

- **矩阵表示**：将模型表示为 $y = X\beta + \varepsilon$

矩阵形式推导

- **矩阵表示**：将模型表示为 $y = X\beta + \varepsilon$
  - X：设计矩阵（包含常数项和所有特征）

矩阵形式推导

- **矩阵表示**：将模型表示为 $y = X\beta + \varepsilon$
  - X：设计矩阵（包含常数项和所有特征）
  - β：参数向量

矩阵形式推导

- **矩阵表示**：将模型表示为 $y = X\beta + \varepsilon$
  - X：设计矩阵（包含常数项和所有特征）
  - β：参数向量
  - y：观测值向量

矩阵形式推导

- **矩阵表示**：将模型表示为 $y = X\beta + \varepsilon$
  - X：设计矩阵（包含常数项和所有特征）
  - β：参数向量
  - y：观测值向量
- **损失函数矩阵形式**:

$$\text{RSS} = (y - X\beta)^\top (y - X\beta)$$

矩阵形式推导

- **矩阵表示**：将模型表示为 $y = X\beta + \varepsilon$
  - X：设计矩阵（包含常数项和所有特征）
  - β：参数向量
  - y：观测值向量
- **损失函数矩阵形式**:

$$\text{RSS} = (y - X\beta)^\top (y - X\beta)$$

- **正规方程解**:

$$\frac{\partial \text{RSS}}{\partial \beta} = -2X^\top y + 2X^\top X\beta = 0$$

矩阵形式推导

- **矩阵表示**：将模型表示为 $y = X\beta + \varepsilon$
  - X：设计矩阵（包含常数项和所有特征）
  - β：参数向量
  - y：观测值向量
- **损失函数矩阵形式**:

$$\text{RSS} = (y - X\beta)^\top (y - X\beta)$$

- **正规方程解**:

$$\frac{\partial \text{RSS}}{\partial \beta} = -2X^\top y + 2X^\top X\beta = 0$$

- **闭式解（当 $X^\top X$ 可逆时）**：

$$\hat{\beta} = (X^\top X)^{-1} X^\top y$$

推导过程关键步骤

- **步骤 1**：展开 RSS 表达式

$$\text{RSS} = y^\top y - 2y^\top X\beta + \beta^\top X^\top X\beta$$

推导过程关键步骤

- **步骤 1**：展开 RSS 表达式

$$\text{RSS} = y^\top y - 2y^\top X\beta + \beta^\top X^\top X\beta$$

- **步骤 2**：对 β 求导

$$\frac{\partial \text{RSS}}{\partial \beta} = -2X^\top y + 2X^\top X\beta$$

推导过程关键步骤

- **步骤 1**：展开 RSS 表达式

$$\text{RSS} = y^\top y - 2y^\top X\beta + \beta^\top X^\top X\beta$$

- **步骤 2**：对 β 求导

$$\frac{\partial \text{RSS}}{\partial \beta} = -2X^\top y + 2X^\top X\beta$$

- **步骤 3**：令导数为 0，得到正规方程

$$X^\top X\beta = X^\top y$$

推导过程关键步骤

- **步骤 1**：展开 RSS 表达式

$$\text{RSS} = y^\top y - 2y^\top X\beta + \beta^\top X^\top X\beta$$

- **步骤 2**：对 β 求导

$$\frac{\partial \text{RSS}}{\partial \beta} = -2X^\top y + 2X^\top X\beta$$

- **步骤 3**：令导数为 0，得到正规方程

$$X^\top X\beta = X^\top y$$

- **步骤 4**：求解参数（要求 $X^\top X$ 满秩）

$$\hat{\beta} = (X^\top X)^{-1}X^\top y$$

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数
- **假设条件**：线性关系、误差项独立同分布、无多重共线性等

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数
- **假设条件**：线性关系、误差项独立同分布、无多重共线性等
- 应用价值:

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数
- **假设条件**：线性关系、误差项独立同分布、无多重共线性等
- **应用价值**:
  - 参数具有明确统计意义，可解释性强

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数
- **假设条件**：线性关系、误差项独立同分布、无多重共线性等
- **应用价值**:
  - 参数具有明确统计意义，可解释性强
  - 计算效率高，适合大规模数据

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数
- **假设条件**：线性关系、误差项独立同分布、无多重共线性等
- **应用价值**:
  - 参数具有明确统计意义，可解释性强
  - 计算效率高，适合大规模数据
  - 为许多高级模型的基础构建块

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数
- **假设条件**：线性关系、误差项独立同分布、无多重共线性等
- **应用价值**:
  - 参数具有明确统计意义，可解释性强
  - 计算效率高，适合大规模数据
  - 为许多高级模型的基础构建块
- **注意事项**:

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数
- **假设条件**：线性关系、误差项独立同分布、无多重共线性等
- **应用价值**：
  - 参数具有明确统计意义，可解释性强
  - 计算效率高，适合大规模数据
  - 为许多高级模型的基础构建块
- **注意事项**：
  - 当 $X^\top X$ 不可逆时需使用正则化方法

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数
- **假设条件**：线性关系、误差项独立同分布、无多重共线性等
- **应用价值**：
  - 参数具有明确统计意义，可解释性强
  - 计算效率高，适合大规模数据
  - 为许多高级模型的基础构建块
- **注意事项**：
  - 当 $X^\top X$ 不可逆时需使用正则化方法
  - 对异常值敏感，需进行数据预处理

核心要点总结

- **数学基础**：最小二乘法通过最小化误差平方和求解最优参数
- **假设条件**：线性关系、误差项独立同分布、无多重共线性等
- **应用价值**:
  - 参数具有明确统计意义，可解释性强
  - 计算效率高，适合大规模数据
  - 为许多高级模型的基础构建块
- **注意事项**:
  - 当 $X^\top X$ 不可逆时需使用正则化方法
  - 对异常值敏感，需进行数据预处理
  - 线性假设需通过残差分析验证

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型

线性模型

线性回归

岭回归与 LASSO 回归

逻辑回归

支持向量机

非线性模型

时间序列模型

正则化：解决过拟合与多重共线性

- **问题背景**：当特征矩阵 $X^\top X$ 接近奇异（不可逆）时，普通最小二乘法（OLS）估计不稳定

正则化：解决过拟合与多重共线性

- **问题背景**：当特征矩阵 $X^\top X$ 接近奇异（不可逆）时，普通最小二乘法（OLS）估计不稳定
- **常见场景**：

正则化：解决过拟合与多重共线性

- **问题背景**：当特征矩阵 $X^\top X$ 接近奇异（不可逆）时，普通最小二乘法（OLS）估计不稳定
- **常见场景**：
  - 特征数量 $>$ 样本数量（高维数据）

正则化：解决过拟合与多重共线性

- **问题背景**：当特征矩阵 $X^\top X$ 接近奇异（不可逆）时，普通最小二乘法（OLS）估计不稳定
- **常见场景**：
  - 特征数量 $>$ 样本数量（高维数据）
  - 特征间高度相关（多重共线性）

正则化：解决过拟合与多重共线性

- **问题背景**：当特征矩阵 $X^\top X$ 接近奇异（不可逆）时，普通最小二乘法（OLS）估计不稳定
- **常见场景**：
  - 特征数量 $>$ 样本数量（高维数据）
  - 特征间高度相关（多重共线性）
  - 数据中存在噪声和异常值

正则化：解决过拟合与多重共线性

- **问题背景**：当特征矩阵 $X^\top X$ 接近奇异（不可逆）时，普通最小二乘法（OLS）估计不稳定
- **常见场景**：
  - 特征数量 $>$ 样本数量（高维数据）
  - 特征间高度相关（多重共线性）
  - 数据中存在噪声和异常值
- **正则化思路**：在损失函数中加入惩罚项，限制模型复杂度

$$J(\beta) = \text{RSS} + \lambda \cdot \text{Penalty}(\beta)$$

正则化：解决过拟合与多重共线性

- **问题背景**：当特征矩阵 $X^\top X$ 接近奇异（不可逆）时，普通最小二乘法（OLS）估计不稳定
- **常见场景**：
  - 特征数量 $>$ 样本数量（高维数据）
  - 特征间高度相关（多重共线性）
  - 数据中存在噪声和异常值
- **正则化思路**：在损失函数中加入惩罚项，限制模型复杂度

$$J(\beta) = \text{RSS} + \lambda \cdot \text{Penalty}(\beta)$$

- **两种主流方法**：岭回归（L2 正则化）vs Lasso 回归（L1 正则化）

岭回归（Ridge Regression）

- **核心思想**：在 OLS 损失函数中加入 L2 正则化项（系数平方和）

岭回归（Ridge Regression）

- **核心思想**：在 OLS 损失函数中加入 L2 正则化项（系数平方和）
- **目标函数**：

$$J(\beta) = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}\beta_j^2$$

岭回归（Ridge Regression）

- **核心思想**：在 OLS 损失函数中加入 L2 正则化项（系数平方和）
- **目标函数**：

$$J(\beta) = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}\beta_j^2$$

- **矩阵形式**：

$$J(\beta) = (Y - X\beta)^\top (Y - X\beta) + \lambda \|\beta\|_2^2$$

岭回归（Ridge Regression）

- **核心思想**：在 OLS 损失函数中加入 L2 正则化项（系数平方和）
- **目标函数**：

$$J(\beta) = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}\beta_j^2$$

- **矩阵形式**：

$$J(\beta) = (Y - X\beta)^\top (Y - X\beta) + \lambda \|\beta\|_2^2$$

- **闭式解（当 $X^\top X$ 可逆时）**：

$$\hat{\beta}^{ridge} = (X^\top X + \lambda I)^{-1} X^\top Y$$

岭回归（Ridge Regression）

- **核心思想**：在 OLS 损失函数中加入 L2 正则化项（系数平方和）
- **目标函数**：

$$J(\beta) = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}\beta_j^2$$

- **矩阵形式**：

$$J(\beta) = (Y - X\beta)^\top (Y - X\beta) + \lambda \|\beta\|_2^2$$

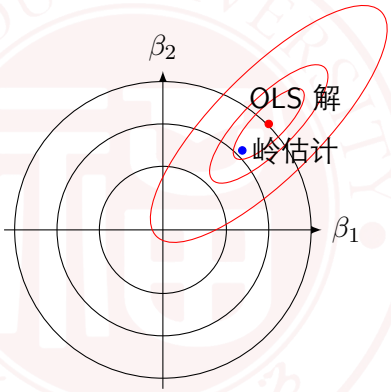
- **闭式解**（当 $X^\top X$ 可逆时）：

$$\hat{\beta}^{ridge} = (X^\top X + \lambda I)^{-1} X^\top Y$$

- **特点**：压缩系数但不置零，稳定估计，处理多重共线性

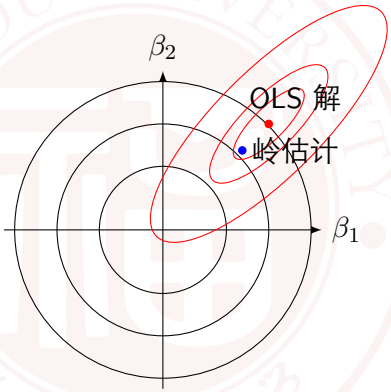
岭回归的几何解释与性质

- **几何解释**：约束区域为圆形/超球体



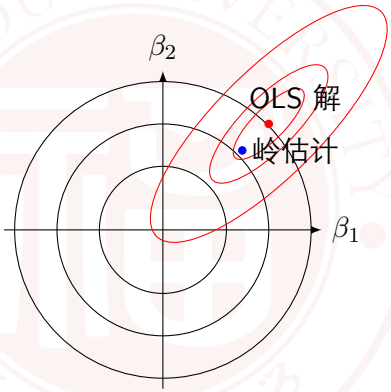
岭回归的几何解释与性质

- **几何解释**：约束区域为圆形/超球体
  - 最优解在椭圆等高线与圆形约束边界相切处



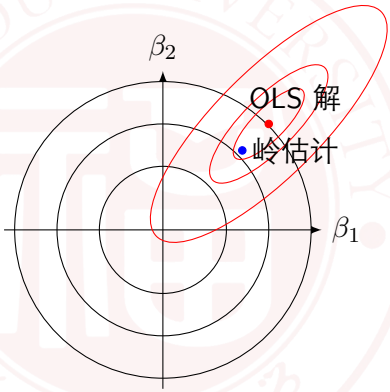
岭回归的几何解释与性质

- **几何解释**：约束区域为圆形/超球体
  - 最优解在椭圆等高线与圆形约束边界相切处
- **系数收缩**：随着 λ 增大，所有系数向零收缩



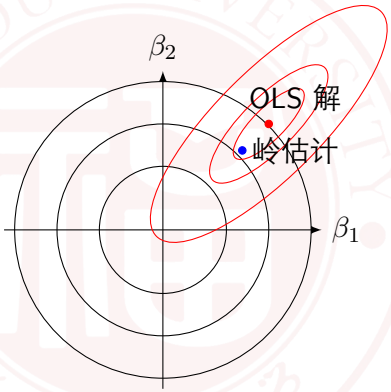
岭回归的几何解释与性质

- **几何解释**：约束区域为圆形/超球体
  - 最优解在椭圆等高线与圆形约束边界相切处
- **系数收缩**：随着 λ 增大，所有系数向零收缩
- **偏差-方差权衡**：



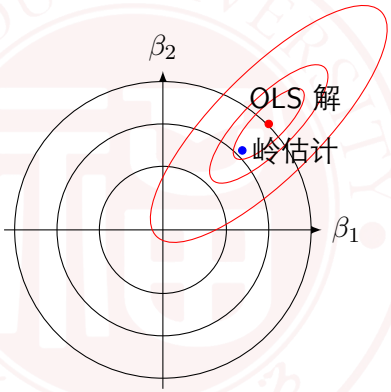
岭回归的几何解释与性质

- **几何解释**：约束区域为圆形/超球体
  - 最优解在椭圆等高线与圆形约束边界相切处
- **系数收缩**：随着 λ 增大，所有系数向零收缩
- **偏差-方差权衡**：
  - $\lambda = 0$：无正则化（OLS 估计）



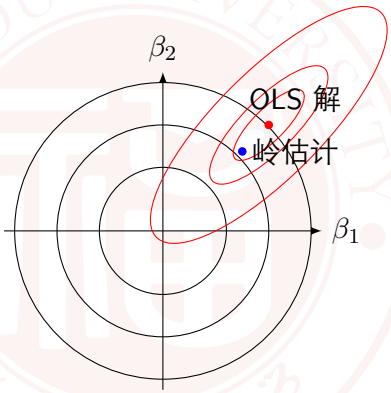
岭回归的几何解释与性质

- **几何解释**：约束区域为圆形/超球体
  - 最优解在椭圆等高线与圆形约束边界相切处
- **系数收缩**：随着 λ 增大，所有系数向零收缩
- **偏差-方差权衡**：
  - $\lambda = 0$：无正则化（OLS 估计）
  - $\lambda \to \infty$：所有系数趋近于零



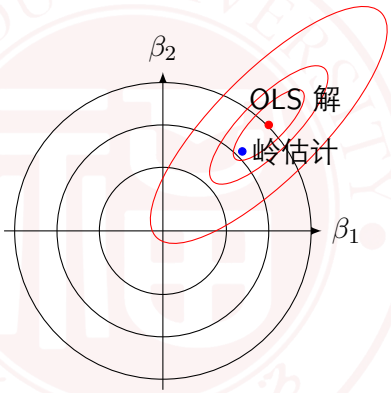
岭回归的几何解释与性质

- **几何解释**：约束区域为圆形/超球体
  - 最优解在椭圆等高线与圆形约束边界相切处
- **系数收缩**：随着 λ 增大，所有系数向零收缩
- **偏差-方差权衡**：
  - $\lambda = 0$：无正则化（OLS 估计）
  - $\lambda \to \infty$：所有系数趋近于零
  - 最优 λ 在两者之间



岭回归的几何解释与性质

- **几何解释**：约束区域为圆形/超球体
  - 最优解在椭圆等高线与圆形约束边界相切处
- **系数收缩**：随着 λ 增大，所有系数向零收缩
- **偏差-方差权衡**：
  - $\lambda = 0$：无正则化（OLS 估计）
  - $\lambda \to \infty$：所有系数趋近于零
  - 最优 λ 在两者之间



岭迹图：通过观察不同 λ 下系数变化选择合适参数

Lasso 回归（Least Absolute Shrinkage and Selection Operator）

- **核心思想**：在 OLS 损失函数中加入 L1 正则化项（系数绝对值之和）

Lasso 回归（Least Absolute Shrinkage and Selection Operator）

- **核心思想**：在 OLS 损失函数中加入 L1 正则化项（系数绝对值之和）
- **目标函数**：

$$J(\beta) = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}|\beta_j|$$

Lasso 回归（Least Absolute Shrinkage and Selection Operator）

- **核心思想**：在 OLS 损失函数中加入 L1 正则化项（系数绝对值之和）
- **目标函数**：

$$J(\beta) = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}|\beta_j|$$

- **矩阵形式**：

$$J(\beta) = (Y - X\beta)^\top (Y - X\beta) + \lambda \|\beta\|_1$$

Lasso 回归（Least Absolute Shrinkage and Selection Operator）

- **核心思想**：在 OLS 损失函数中加入 L1 正则化项（系数绝对值之和）
- **目标函数**：

$$J(\beta) = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}|\beta_j|$$

- **矩阵形式**：

$$J(\beta) = (Y - X\beta)^\top (Y - X\beta) + \lambda \|\beta\|_1$$

- **特点**：产生稀疏解，自动进行特征选择

Lasso 回归（Least Absolute Shrinkage and Selection Operator）

- **核心思想**：在 OLS 损失函数中加入 L1 正则化项（系数绝对值之和）
- **目标函数**：

$$J(\beta) = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}|\beta_j|$$

- **矩阵形式**：

$$J(\beta) = (Y - X\beta)^\top (Y - X\beta) + \lambda \|\beta\|_1$$

- **特点**：产生稀疏解，自动进行特征选择
- **几何解释**：约束区域为菱形/多面体，解容易出现在顶点上（某些系数为零）

Lasso 回归（Least Absolute Shrinkage and Selection Operator）

- **核心思想**：在 OLS 损失函数中加入 L1 正则化项（系数绝对值之和）
- **目标函数**：

$$J(\beta) = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}|\beta_j|$$

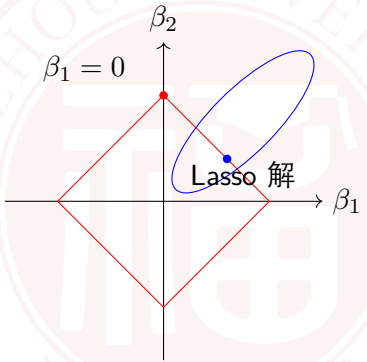
- **矩阵形式**：

$$J(\beta) = (Y - X\beta)^\top (Y - X\beta) + \lambda \|\beta\|_1$$

- **特点**：产生稀疏解，自动进行特征选择
- **几何解释**：约束区域为菱形/多面体，解容易出现在顶点上（某些系数为零）
- **求解方法**：坐标下降法、最小角回归（LARS）等

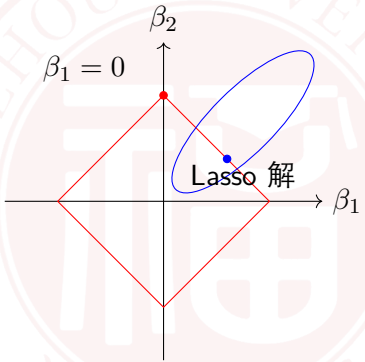
Lasso 回归的稀疏性与特征选择

- **稀疏性效应**：L1 正则化倾向于将不重要特征的系数压缩至零



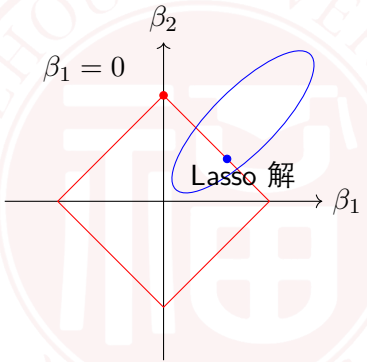
Lasso 回归的稀疏性与特征选择

- **稀疏性效应**：L1 正则化倾向于将不重要特征的系数压缩至零
- **特征选择**：零系数对应特征被自动筛选掉



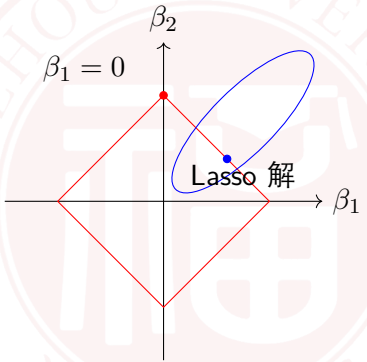
Lasso 回归的稀疏性与特征选择

- **稀疏性效应**：L1 正则化倾向于将不重要特征的系数压缩至零
- **特征选择**：零系数对应特征被自动筛选掉
- **几何解释**：菱形约束的尖角导致稀疏解



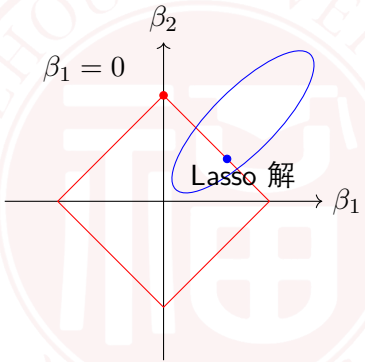
Lasso 回归的稀疏性与特征选择

- **稀疏性效应**：L1 正则化倾向于将不重要特征的系数压缩至零
- **特征选择**：零系数对应特征被自动筛选掉
- **几何解释**：菱形约束的尖角导致稀疏解
- **参数调节**：



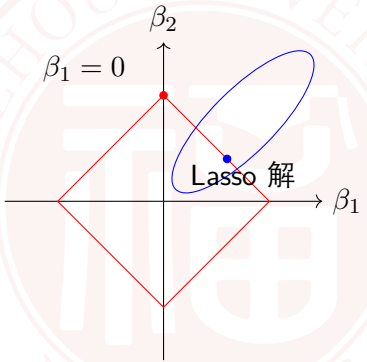
Lasso 回归的稀疏性与特征选择

- **稀疏性效应**：L1 正则化倾向于将不重要特征的系数压缩至零
- **特征选择**：零系数对应特征被自动筛选掉
- **几何解释**：菱形约束的尖角导致稀疏解
- **参数调节**：
  - λ 较小：保留更多特征



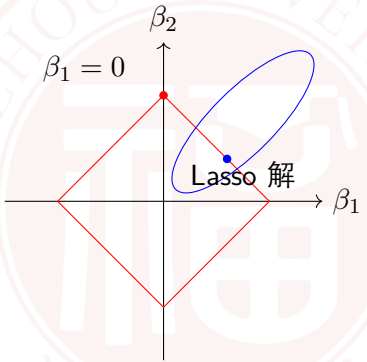
Lasso 回归的稀疏性与特征选择

- **稀疏性效应**：L1 正则化倾向于将不重要特征的系数压缩至零
- **特征选择**：零系数对应特征被自动筛选掉
- **几何解释**：菱形约束的尖角导致稀疏解
- **参数调节**：
  - λ 较小：保留更多特征
  - λ 较大：更严格的筛选，模型更稀疏



Lasso 回归的稀疏性与特征选择

- **稀疏性效应**：L1 正则化倾向于将不重要特征的系数压缩至零
- **特征选择**：零系数对应特征被自动筛选掉
- **几何解释**：菱形约束的尖角导致稀疏解
- **参数调节**：
  - λ 较小：保留更多特征
  - λ 较大：更严格的筛选，模型更稀疏



应用优势：特别适用于高维数据（特征数 $p >$ 样本数 n）的场景

岭回归 vs Lasso 回归：核心区别对比

对比维度	岭回归（L2 正则化）	Lasso 回归（L1 正则化）
正则化项	$\lambda\|\beta\|_2^2 = \lambda\sum\beta_j^2$	$\lambda\|\beta\|_1 = \lambda\sum\|\beta_j\|$
解的性质	光滑，系数连续收缩	稀疏，部分系数精确为零
特征选择	无，保留所有特征	有，自动进行特征选择
几何约束	圆形/超球体	菱形/多面体
计算复杂度	有解析解，计算简单	需迭代求解，计算复杂
适用场景	特征都相关，多重共线性强	特征稀疏，需要模型简化

总结

- **岭回归**：通过 L2 正则化处理多重共线性，提高模型稳定性

总结

- **岭回归**：通过 L2 正则化处理多重共线性，提高模型稳定性
- **Lasso 回归**：通过 L1 正则化实现特征选择，生成稀疏模型

总结

- **岭回归**：通过 L2 正则化处理多重共线性，提高模型稳定性
- **Lasso 回归**：通过 L1 正则化实现特征选择，生成稀疏模型
- **关键区别**：约束形式不同导致解的性质差异（连续 vs 稀疏）

总结

- **岭回归**：通过 L2 正则化处理多重共线性，提高模型稳定性
- **Lasso 回归**：通过 L1 正则化实现特征选择，生成稀疏模型
- **关键区别**：约束形式不同导致解的性质差异（连续 vs 稀疏）
- **选择依据**：根据数据特征和建模目标选择合适方法

总结

- **岭回归**：通过 L2 正则化处理多重共线性，提高模型稳定性
- **Lasso 回归**：通过 L1 正则化实现特征选择，生成稀疏模型
- **关键区别**：约束形式不同导致解的性质差异（连续 vs 稀疏）
- **选择依据**：根据数据特征和建模目标选择合适方法
- **实践要点**：正则化参数调优至关重要，推荐交叉验证方法

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
  - 线性回归
  - 岭回归与 LASSO 回归
  - 逻辑回归
  - 支持向量机
- 非线性模型
- 时间序列模型

逻辑回归基本概念

- **定义**：逻辑回归是一种用于解决二分类问题的统计学习方法，通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间

逻辑回归基本概念

- **定义**：逻辑回归是一种用于解决二分类问题的统计学习方法，通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间
- **核心特点**:

逻辑回归基本概念

- **定义**：逻辑回归是一种用于解决二分类问题的统计学习方法，通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间
- **核心特点**:
  - 输出具有概率意义，提供样本属于某类别的概率估计

逻辑回归基本概念

- **定义**：逻辑回归是一种用于解决二分类问题的统计学习方法，通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间
- **核心特点**:
  - 输出具有概率意义，提供样本属于某类别的概率估计
  - 虽然是“回归”但实际用于分类任务

逻辑回归基本概念

- **定义**：逻辑回归是一种用于解决二分类问题的统计学习方法，通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间
- **核心特点**:
  - 输出具有概率意义，提供样本属于某类别的概率估计
  - 虽然是“回归”但实际用于分类任务
  - 模型参数具有强可解释性

逻辑回归基本概念

- **定义**：逻辑回归是一种用于解决二分类问题的统计学习方法，通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间
- **核心特点**:
  - 输出具有概率意义，提供样本属于某类别的概率估计
  - 虽然是“回归”但实际用于分类任务
  - 模型参数具有强可解释性
- **与线性回归的区别**:

逻辑回归基本概念

- **定义**：逻辑回归是一种用于解决二分类问题的统计学习方法，通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间
- **核心特点**:
  - 输出具有概率意义，提供样本属于某类别的概率估计
  - 虽然是“回归”但实际用于分类任务
  - 模型参数具有强可解释性
- **与线性回归的区别**:
  - 线性回归输出连续值，逻辑回归输出离散类别

逻辑回归基本概念

- **定义**：逻辑回归是一种用于解决二分类问题的统计学习方法，通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间
- **核心特点**:
  - 输出具有概率意义，提供样本属于某类别的概率估计
  - 虽然是“回归”但实际用于分类任务
  - 模型参数具有强可解释性
- **与线性回归的区别**:
  - 线性回归输出连续值，逻辑回归输出离散类别
  - 逻辑回归通过 Sigmoid 函数实现概率映射

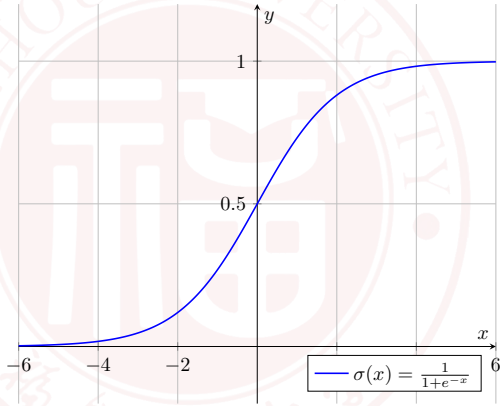
逻辑回归基本概念

- **定义**：逻辑回归是一种用于解决二分类问题的统计学习方法，通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间
- **核心特点**:
  - 输出具有概率意义，提供样本属于某类别的概率估计
  - 虽然是“回归”但实际用于分类任务
  - 模型参数具有强可解释性
- **与线性回归的区别**:
  - 线性回归输出连续值，逻辑回归输出离散类别
  - 逻辑回归通过 Sigmoid 函数实现概率映射
- **应用领域**：医疗诊断、金融风控、推荐系统等

Sigmoid 函数与数学模型

- **Sigmoid** 函数定义:

$$g(z) = \frac{1}{1 + e^{-z}}$$



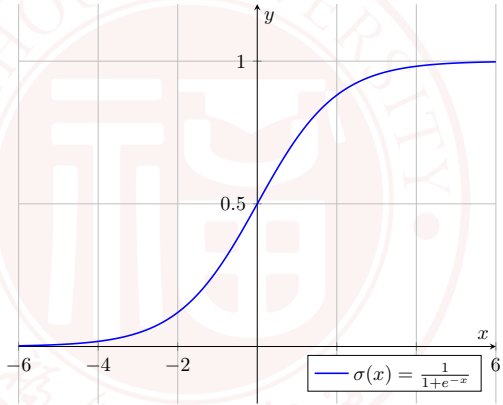
Sigmoid 函数与数学模型

- **Sigmoid 函数定义**:

$$g(z) = \frac{1}{1 + e^{-z}}$$

- **逻辑回归模型**:

$$P(Y = 1|X) = g(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)$$



Sigmoid 函数与数学模型

- **Sigmoid 函数定义**:

$$g(z) = \frac{1}{1 + e^{-z}}$$

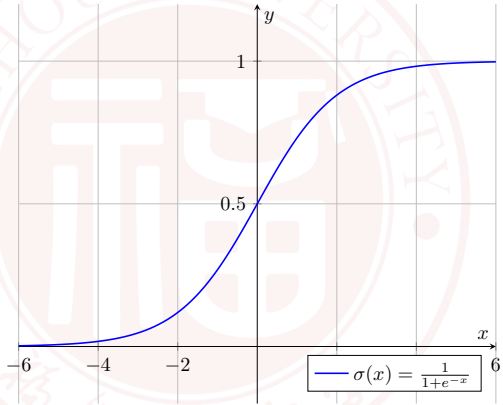
- **逻辑回归模型**:

$$P(Y = 1|X) = g(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)$$

- **发生比与对数发生比**:

$$\frac{P(Y = 1|X)}{1 - P(Y = 1|X)} = e^{\beta^T X}$$

$$\ln\left(\frac{P(Y = 1|X)}{1 - P(Y = 1|X)}\right) = \beta^T X$$



Sigmoid 函数与数学模型

- **Sigmoid 函数定义**:

$$g(z) = \frac{1}{1 + e^{-z}}$$

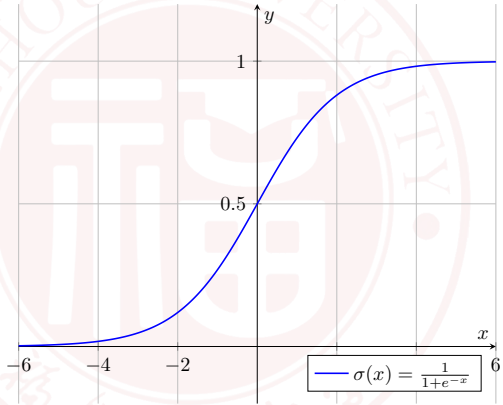
- **逻辑回归模型**:

$$P(Y = 1|X) = g(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)$$

- **发生比与对数发生比**:

$$\frac{P(Y = 1|X)}{1 - P(Y = 1|X)} = e^{\beta^T X}$$

$$\ln\left(\frac{P(Y = 1|X)}{1 - P(Y = 1|X)}\right) = \beta^T X$$



损失函数与优化目标

- **交叉熵损失函数（Cross Entropy）**：

$$\mathcal{L}(\beta) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

其中，y_i 是真实值，$\hat{y}_i$ 是分类概率。

损失函数与优化目标

- **交叉熵损失函数（Cross Entropy）**：

$$\mathcal{L}(\beta) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

其中，y_i 是真实值，$\hat{y}_i$ 是分类概率。

- **极大似然估计视角**:

损失函数与优化目标

- **交叉熵损失函数（Cross Entropy）**：

$$\mathcal{L}(\beta) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

其中，y_i 是真实值，$\hat{y}_i$ 是分类概率。

- **极大似然估计视角**:
  - 似然函数：$\mathcal{L}(\beta) = \prod_{i=1}^{n} P(Y=1|X_i)^{y_i}[1-P(Y=1|X_i)]^{1-y_i}$

损失函数与优化目标

- **交叉熵损失函数（Cross Entropy）**：

$$\mathcal{L}(\beta) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

其中，y_i 是真实值，$\hat{y}_i$ 是分类概率。

- **极大似然估计视角**:
  - 似然函数：$\mathcal{L}(\beta) = \prod_{i=1}^{n} P(Y=1|X_i)^{y_i}[1-P(Y=1|X_i)]^{1-y_i}$
  - 对数似然函数：$\ell(\beta) = \sum_{i=1}^{n}[y_i \log(P_i) + (1-y_i)\log(1-P_i)]$

损失函数与优化目标

- **交叉熵损失函数（Cross Entropy）**：

$$\mathcal{L}(\beta) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

其中，y_i 是真实值，$\hat{y}_i$ 是分类概率。

- **极大似然估计视角**:
  - 似然函数：$\mathcal{L}(\beta) = \prod_{i=1}^{n} P(Y=1|X_i)^{y_i}[1-P(Y=1|X_i)]^{1-y_i}$
  - 对数似然函数：$\ell(\beta) = \sum_{i=1}^{n}[y_i \log(P_i) + (1-y_i)\log(1-P_i)]$
- **Sigmoid 函数求导**:

$$g'(z) = g(z)(1-g(z))$$

这一性质使得梯度计算非常高效.

多分类逻辑回归

- **One-vs-Rest (OvR) 策略**:

多分类逻辑回归

- **One-vs-Rest (OvR) 策略**:
  - 为每个类别训练一个二分类器

多分类逻辑回归

- **One-vs-Rest (OvR) 策略**:
  - 为每个类别训练一个二分类器
  - 选择概率最高的类别作为预测结果

多分类逻辑回归

- **One-vs-Rest (OvR) 策略**:
  - 为每个类别训练一个二分类器
  - 选择概率最高的类别作为预测结果
  - 举例：66% 的概率是 A，90% 的概率是 B，则考虑

多分类逻辑回归

- **One-vs-Rest (OvR) 策略**:
  - 为每个类别训练一个二分类器
  - 选择概率最高的类别作为预测结果
  - 举例：66% 的概率是 A，90% 的概率是 B，则考虑
- **One-vs-One (OvO) 策略**:

多分类逻辑回归

- **One-vs-Rest (OvR) 策略**:
  - 为每个类别训练一个二分类器
  - 选择概率最高的类别作为预测结果
  - 举例：66% 的概率是 A，90% 的概率是 B，则考虑
- **One-vs-One (OvO) 策略**:
  - 为每两个类别训练一个二分类器。

多分类逻辑回归

- **One-vs-Rest (OvR) 策略**:
  - 为每个类别训练一个二分类器
  - 选择概率最高的类别作为预测结果
  - 举例：66% 的概率是 A，90% 的概率是 B，则考虑
- **One-vs-One (OvO) 策略**:
  - 为每两个类别训练一个二分类器。
  - 最终以投票的形式决定最终的类别。

多分类逻辑回归

- **One-vs-Rest (OvR) 策略**:
  - 为每个类别训练一个二分类器
  - 选择概率最高的类别作为预测结果
  - 举例：66% 的概率是 A，90% 的概率是 B，则考虑
- **One-vs-One (OvO) 策略**:
  - 为每两个类别训练一个二分类器。
  - 最终以投票的形式决定最终的类别。
- **Softmax 回归**:

$$P(Y = k|X) = \frac{e^{\beta_k^T X}}{\sum_{j=1}^{K} e^{\beta_j^T X}}$$

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

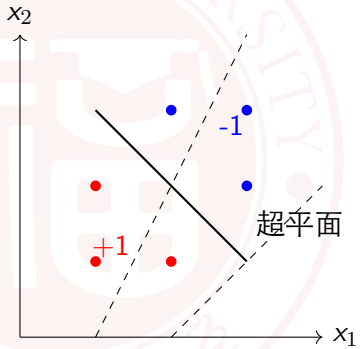
4 传统机器学习模型
- 线性模型
  - 线性回归
  - 岭回归与 LASSO 回归
  - 逻辑回归
  - 支持向量机
- 非线性模型
- 时间序列模型

什么是支持向量机（SVM）？

- **定义**：一种监督学习的二分类模型，基本模型是定义在特征空间上的**间隔最大的线性分类器**。

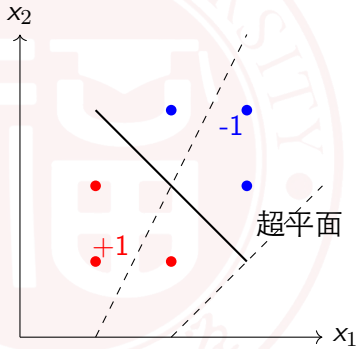
什么是支持向量机（SVM）？

- **定义**：一种监督学习的二分类模型，基本模型是定义在特征空间上的**间隔最大的线性分类器**。
- **核心目标**：寻找一个超平面，将不同类别的数据分开，且使间隔 (margin) 最大化。



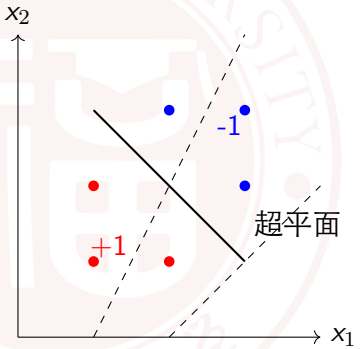
什么是支持向量机（SVM）？

- **定义**：一种监督学习的二分类模型，基本模型是定义在特征空间上的**间隔最大的线性分类器**。
- **核心目标**：寻找一个超平面，将不同类别的数据分开，且使间隔 (margin) 最大化。
- **独特优势**:



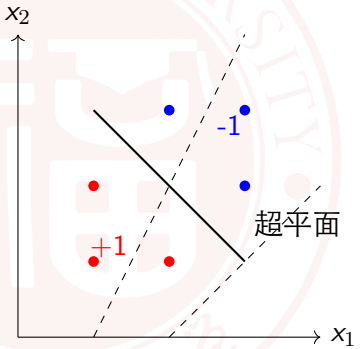
什么是支持向量机（SVM）？

- **定义**：一种监督学习的二分类模型，基本模型是定义在特征空间上的**间隔最大的线性分类器**。
- **核心目标**：寻找一个超平面，将不同类别的数据分开，且使间隔 (margin) 最大化。
- **独特优势**:
  - 适用于小样本学习。



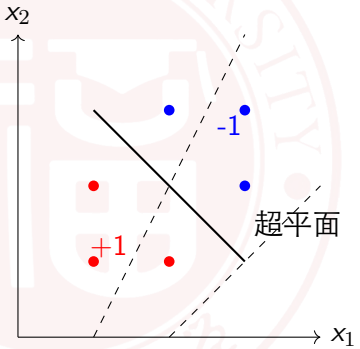
什么是支持向量机（SVM）？

- **定义**：一种监督学习的二分类模型，基本模型是定义在特征空间上的**间隔最大的线性分类器**。
- **核心目标**：寻找一个超平面，将不同类别的数据分开，且使间隔 (margin) 最大化。
- **独特优势**:
  - 适用于小样本学习。
  - 理论基础牢固，泛化能力强。



什么是支持向量机（SVM）？

- **定义**：一种监督学习的二分类模型，基本模型是定义在特征空间上的**间隔最大的线性分类器**。
- **核心目标**：寻找一个超平面，将不同类别的数据分开，且使间隔 (margin) 最大化。
- **独特优势**:
  - 适用于小样本学习。
  - 理论基础牢固，泛化能力强。
  - 通过核技巧处理非线性问题。



函数间隔与几何间隔

- **函数间隔 (Functional Margin)**:

$$\hat{\gamma}_i = y_i(\mathbf{w}^T\mathbf{x}_i + b)$$

表示分类的正确性和确信度，但会随 $\mathbf{w}$ 和 b 的缩放而改变。

函数间隔与几何间隔

- 函数间隔 **(Functional Margin)**:

$$\hat{\gamma}_i = y_i(\mathbf{w}^T\mathbf{x}_i + b)$$

表示分类的正确性和确信度，但会随 $\mathbf{w}$ 和 b 的缩放而改变。

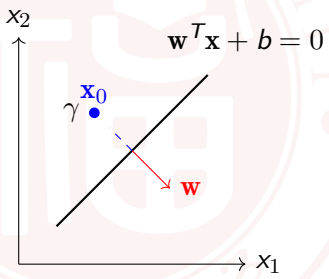
- 几何间隔 **(Geometric Margin)**

$$\gamma_i = \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|} = \frac{\hat{\gamma}_i}{\|\mathbf{w}\|}$$

是点到超平面的实际距离，具有尺度不变性

几何间隔的几何推导

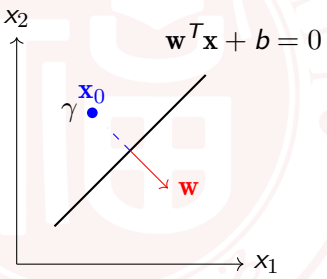
- 超平面法向量：$\mathbf{w}$



几何间隔的几何推导

- 超平面法向量：$\mathbf{w}$
- 点 $\mathbf{x}_0$ 到超平面的距离：

$$\gamma = \frac{|\mathbf{w}^T\mathbf{x}_0 + b|}{\|\mathbf{w}\|}$$



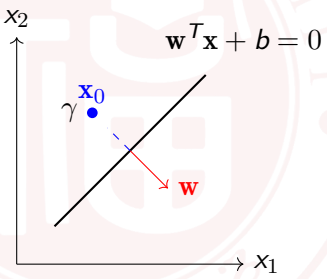
几何间隔的几何推导

- 超平面法向量：$\mathbf{w}$
- 点 $\mathbf{x}_0$ 到超平面的距离：

$$\gamma = \frac{|\mathbf{w}^T\mathbf{x}_0 + b|}{\|\mathbf{w}\|}$$

- 考虑分类正确性：

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) > 0$$



几何间隔的几何推导

- 超平面法向量：$\mathbf{w}$
- 点 $\mathbf{x}_0$ 到超平面的距离：

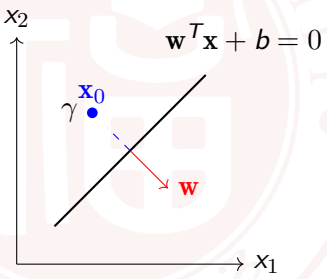
$$\gamma = \frac{|\mathbf{w}^T\mathbf{x}_0 + b|}{\|\mathbf{w}\|}$$

- 考虑分类正确性：

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) > 0$$

- 因此几何间隔为：

$$\gamma_i = \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|}$$



最大间隔的数学表述

- **目标**：找到使几何间隔最大的超平面

$$\max_{\mathbf{w},b} \gamma = \max_{\mathbf{w},b} \min_{i=1,\dots,m} \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|}$$

最大间隔的数学表述

- **目标**：找到使几何间隔最大的超平面

$$\max_{\mathbf{w},b} \gamma = \max_{\mathbf{w},b} \min_{i=1,\dots,m} \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|}$$

- **简化处理**：令函数间隔 $\hat{\gamma} = 1$，问题转化为：

$$\max_{\mathbf{w},b} \frac{1}{\|\mathbf{w}\|} \quad \text{s.t.} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1,\ i = 1, \dots, m$$

最大间隔的数学表述

- **目标**：找到使几何间隔最大的超平面

$$\max_{\mathbf{w},b} \gamma = \max_{\mathbf{w},b} \min_{i=1,\dots,m} \frac{y_i(\mathbf{w}^T\mathbf{x}_i + b)}{\|\mathbf{w}\|}$$

- **简化处理**：令函数间隔 $\hat{\gamma} = 1$，问题转化为：

$$\max_{\mathbf{w},b} \frac{1}{\|\mathbf{w}\|} \quad \text{s.t.} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1,\ i = 1, \dots, m$$

- **等价形式**

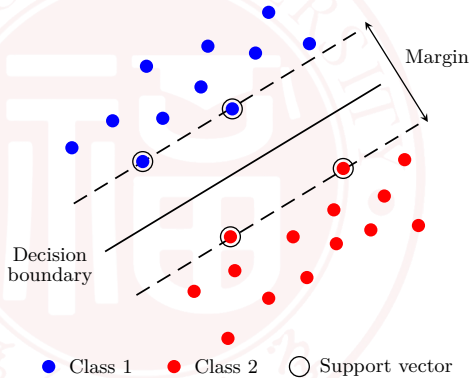
$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$

最大间隔原理与支持向量

- **间隔 (Margin)**：超平面到最近数据点的距离。

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|}$$

- **支持向量**：距离超平面最近的数据点，决定超平面位置。



拉格朗日对偶问题：步骤 1

- 原始优化问题：

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b) \leq 0,\ i = 1, \ldots, m$$

拉格朗日对偶问题：步骤 1

- **原始优化问题**：

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b) \le 0,\ i = 1, \ldots, m$$

- **拉格朗日函数**：

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{m} \alpha_i[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1]$$

其中 $\alpha_i \ge 0$ 为拉格朗日乘子

拉格朗日对偶问题：步骤 1

- **原始优化问题**：

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b) \le 0,\ i = 1, \ldots, m$$

- **拉格朗日函数**：

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{m} \alpha_i[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1]$$

其中 $\alpha_i \ge 0$ 为拉格朗日乘子

- **KKT 条件**：

$$\alpha_i \ge 0$$

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 \ge 0$$

$$\alpha_i[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1] = 0$$

拉格朗日对偶问题：步骤 2

- **对偶问题**：先求 $L(\mathbf{w}, b, \boldsymbol{\alpha})$ 关于 $\mathbf{w}, b$ 的极小值。

拉格朗日对偶问题：步骤 2

- **对偶问题**：先求 $L(\mathbf{w}, b, \boldsymbol{\alpha})$ 关于 $\mathbf{w}, b$ 的极小值。
- **求偏导数** [1](@ref)：

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i = 0$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{m} \alpha_i y_i = 0$$

拉格朗日对偶问题：步骤 2

- **对偶问题**：先求 $L(\mathbf{w}, b, \boldsymbol{\alpha})$ 关于 $\mathbf{w}, b$ 的极小值。
- **求偏导数** [1](@ref)：

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i = 0$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{m} \alpha_i y_i = 0$$

- **得到关系**：

$$\mathbf{w} = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i$$

$$\sum_{i=1}^{m} \alpha_i y_i = 0$$

拉格朗日对偶问题：步骤 3

- **代入拉格朗日函数**：

$$
\begin{aligned}
L(\mathbf{w}, b, \boldsymbol{\alpha}) &= \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{m} \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] \\
&= \frac{1}{2}\mathbf{w}^T\mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{w}^T \mathbf{x}_i - b\sum_{i=1}^{m} \alpha_i y_i + \sum_{i=1}^{m} \alpha_i
\end{aligned}
$$

拉格朗日对偶问题：步骤 3

- 代入拉格朗日函数：

$$
\begin{aligned}
L(\mathbf{w}, b, \boldsymbol{\alpha}) &= \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{m} \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] \\
&= \frac{1}{2}\mathbf{w}^T\mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{w}^T \mathbf{x}_i - b\sum_{i=1}^{m} \alpha_i y_i + \sum_{i=1}^{m} \alpha_i
\end{aligned}
$$

- 利用关系式化简：

$$
L(\boldsymbol{\alpha}) = \sum_{i=1}^{m} \alpha_i - \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j
$$

最终对偶问题与 KKT 条件

- **对偶优化问题**:

$$\max_{\boldsymbol{\alpha}} \quad \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$\text{s.t.} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0, \ i = 1, \ldots, m$$

最终对偶问题与 KKT 条件

- **对偶优化问题**：

$$\max_{\boldsymbol{\alpha}} \quad \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$\text{s.t.} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0, \ i = 1, \ldots, m$$

- **KKT 互补条件**：

$$\alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0, \quad i = 1, \ldots, m$$

最终对偶问题与 KKT 条件

- **对偶优化问题**：

$$\max_{\boldsymbol{\alpha}} \quad \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$\text{s.t.} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0, \ i = 1, \ldots, m$$

- **KKT 互补条件**：

$$\alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0, \quad i = 1, \ldots, m$$

- **支持向量的判断**：

最终对偶问题与 KKT 条件

- **对偶优化问题**：

$$\max_{\boldsymbol{\alpha}} \quad \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$\text{s.t.} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0, \ i = 1, \ldots, m$$

- **KKT 互补条件**：

$$\alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0, \quad i = 1, \ldots, m$$

- **支持向量的判断**：
  - 若 $\alpha_i > 0$，则 $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$，该点为支持向量

最终对偶问题与 KKT 条件

- **对偶优化问题**：

$$\max_{\boldsymbol{\alpha}} \quad \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$\text{s.t.} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0, \ i = 1, \ldots, m$$

- **KKT 互补条件**：

$$\alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0, \quad i = 1, \ldots, m$$

- **支持向量的判断**：
  - 若 $\alpha_i > 0$，则 $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$，该点为支持向量
  - 若 $\alpha_i = 0$，则该点不是支持向量，对模型无影响

最终决策函数

- 权重向量表示：

$$\mathbf{w}^* = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i$$

最终决策函数

- **权重向量表示**：

$$\mathbf{w}^* = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i$$

- **偏置项计算**：对于任意支持向量 $(\mathbf{x}_s, y_s)$，

$$b^* = y_s - \mathbf{w}^{*T} \mathbf{x}_s$$

最终决策函数

- **权重向量表示**：

$$\mathbf{w}^* = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i$$

- **偏置项计算**：对于任意支持向量 $(\mathbf{x}_s, y_s)$，

$$b^* = y_s - \mathbf{w}^{*T} \mathbf{x}_s$$

- **最终决策函数**：

$$f(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b \right)$$

最终决策函数

- **权重向量表示**：

$$\mathbf{w}^* = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i$$

- **偏置项计算**：对于任意支持向量 $(\mathbf{x}_s, y_s)$，

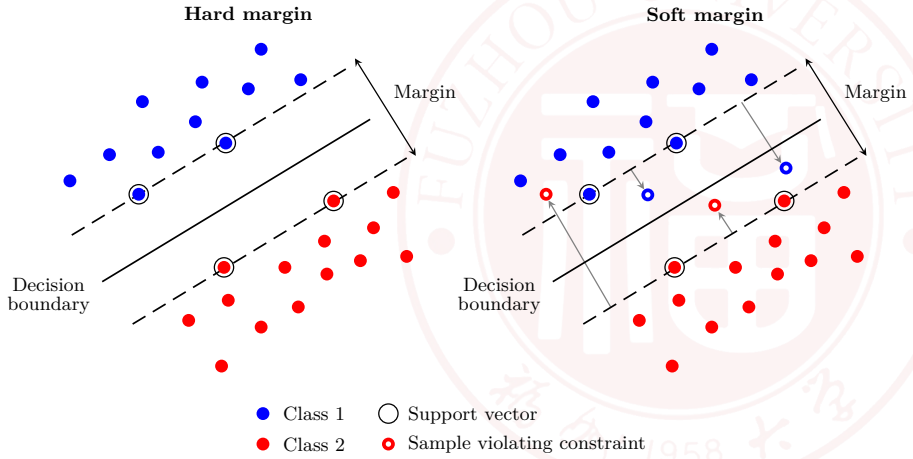
$$b^* = y_s - \mathbf{w}^{*T} \mathbf{x}_s$$

- **最终决策函数**：

$$f(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b \right)$$

- **稀疏性**：只有支持向量对应的 $\alpha_i > 0$，其他样本的 $\alpha_i = 0$

软间隔 SVM


Hard margin
Margin
Decision boundary
Soft margin
Margin
Decision boundary
Class 1
Class 2
Support vector
Sample violating constraint

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

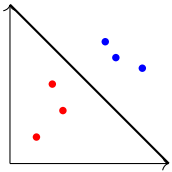
3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
  - 决策树
  - 集成学习与随机森林
  - AdaBoost
  - Gradient Boosting
  - XGBoost
  - LightGBM
  - 核方法

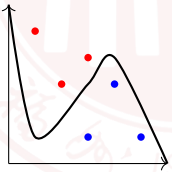
为什么需要非线性模型?

- **现实世界的复杂性**：大多数真实世界的关系是非线性的

线性决策边界



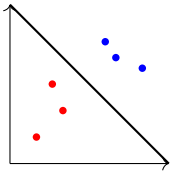
非线性决策边界



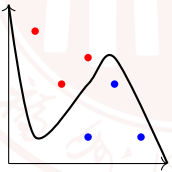
为什么需要非线性模型?

- **现实世界的复杂性**：大多数真实世界的关系是非线性的
- **线性模型的局限**：无法捕捉特征间的交互作用和复杂模式

线性决策边界



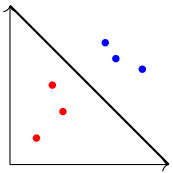
非线性决策边界



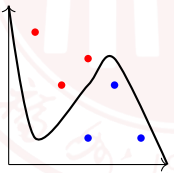
为什么需要非线性模型?

- **现实世界的复杂性**：大多数真实世界的关系是非线性的
- **线性模型的局限**：无法捕捉特征间的交互作用和复杂模式
- **非线性模型优势**：

线性决策边界



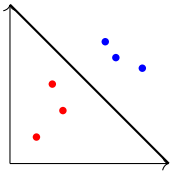
非线性决策边界



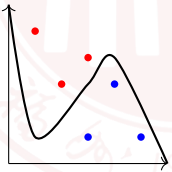
为什么需要非线性模型?

- **现实世界的复杂性**：大多数真实世界的关系是非线性的
- **线性模型的局限**：无法捕捉特征间的交互作用和复杂模式
- **非线性模型优势**：
  - 处理曲线边界和复杂决策区域

线性决策边界



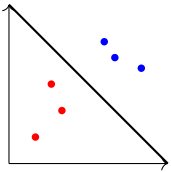
非线性决策边界



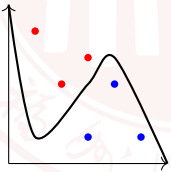
为什么需要非线性模型?

- **现实世界的复杂性**：大多数真实世界的关系是非线性的
- **线性模型的局限**：无法捕捉特征间的交互作用和复杂模式
- **非线性模型优势**：
  - 处理曲线边界和复杂决策区域
  - 学习特征间的高阶交互

线性决策边界



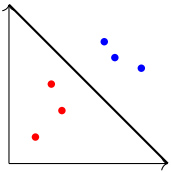
非线性决策边界



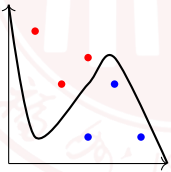
为什么需要非线性模型?

- **现实世界的复杂性**：大多数真实世界的关系是非线性的
- **线性模型的局限**：无法捕捉特征间的交互作用和复杂模式
- **非线性模型优势**：
  - 处理曲线边界和复杂决策区域
  - 学习特征间的高阶交互
  - 适应非均匀分布的数据结构

线性决策边界



非线性决策边界



1 数据挖掘与机器学习的基本概念

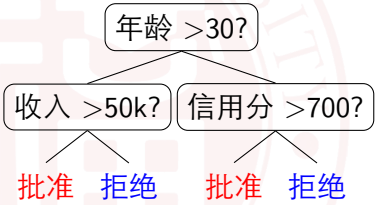
2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
  - 决策树
  - 集成学习与随机森林
  - AdaBoost
  - Gradient Boosting
  - XGBoost
  - LightGBM
  - 核方法

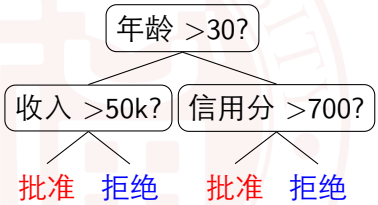
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。



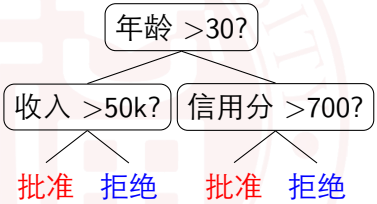
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.



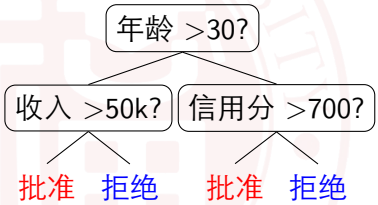
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.



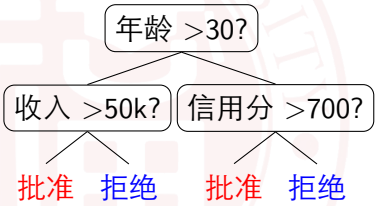
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.
- **构建过程**：



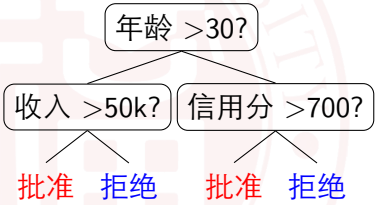
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.
- **构建过程**：
  - 选择最佳分裂特征和阈值



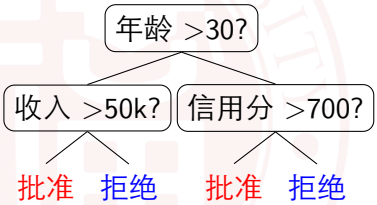
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.
- **构建过程**：
  - 选择最佳分裂特征和阈值
  - 递归构建子树



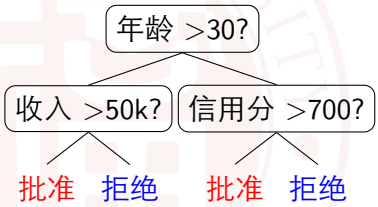
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.
- **构建过程**：
  - 选择最佳分裂特征和阈值
  - 递归构建子树
  - 直到满足停止条件



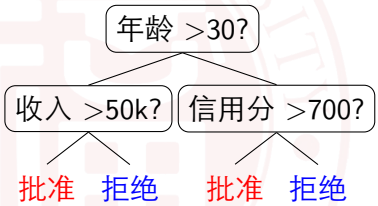
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.
- **构建过程**
- **分裂准则**：



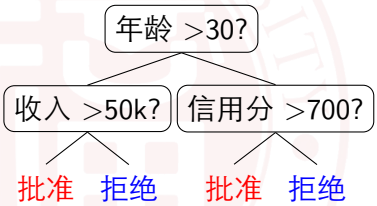
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.
- **构建过程**
- **分裂准则**：
  - 信息增益（Information gain）



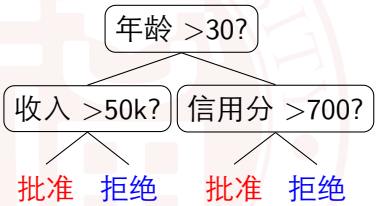
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.
- **构建过程**
- **分裂准则**：
  - 信息增益（Information gain）
  - 基尼不纯度（Gini impurity）



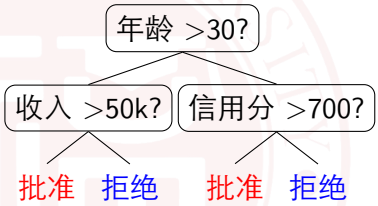
决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.
- **构建过程**
- **分裂准则**：
  - 信息增益（Information gain）
  - 基尼不纯度（Gini impurity）
  - 均方误差（回归树）



决策树：基础非线性模型

- **起源**：1966 年 E.B.Hunt 等人提出决策树概念。
- **核心思想**：通过一系列规则划分特征空间来实现回归/分类.
- **特点**：树形结构；内部节点 = 属性测试，分支 = 测试输出，叶节点 = 类别.
- **构建过程**
- **分裂准则**



优缺点

优点：直观易解释，无需特征缩放，处理混合类型数据
缺点：容易过拟合，对数据微小变化敏感

信息增益：基于信息论的方法——ID3 的基础

- **核心概念**：信息熵 (Entropy) 衡量系统的不确定性。

信息增益：基于信息论的方法——ID3 的基础

- **核心概念**：信息熵 (Entropy) 衡量系统的不确定性。
- **熵的计算公式**：

$$H(X) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

信息增益：基于信息论的方法——ID3 的基础

- **核心概念**：信息熵 (Entropy) 衡量系统的不确定性。
- **熵的计算公式**：

$$H(X) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

- **值域**：[0, 1]

信息增益：基于信息论的方法——ID3 的基础

- **核心概念**：信息熵 (Entropy) 衡量系统的不确定性。
- **熵的计算公式**：

$$H(X) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

- **值域**：[0, 1]
  - 0：完全纯净（无不确定性）

信息增益：基于信息论的方法——ID3 的基础

- **核心概念**：信息熵 (Entropy) 衡量系统的不确定性。
- **熵的计算公式**：

$$H(X) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

- **值域**：[0, 1]
  - 0：完全纯净（无不确定性）
  - 1：最大不确定性（二分类均匀分布）

信息增益：基于信息论的方法——ID3 的基础

- **核心概念**：信息熵 (Entropy) 衡量系统的不确定性。
- **熵的计算公式**：

$$H(X) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

- **值域**：[0, 1]
  - 0：完全纯净（无不确定性）
  - 1：最大不确定性（二分类均匀分布）
- **信息增益**：分裂前后熵的减少量

$$IG = H_{\text{parent}} - \sum_{i=1}^{m} \frac{n_i}{n} H_{\text{child}_i}$$

信息增益：基于信息论的方法——ID3 的基础

- **核心概念**：信息熵 (Entropy) 衡量系统的不确定性。
- **熵的计算公式**：

$$H(X) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

- **值域**：[0, 1]
  - 0：完全纯净（无不确定性）
  - 1：最大不确定性（二分类均匀分布）
- **信息增益**：分裂前后熵的减少量

$$IG = H_{\text{parent}} - \sum_{i=1}^{m} \frac{n_i}{n} H_{\text{child}_i}$$

- **分裂准则**：选择信息增益最大的特征和分裂点

示例计算

- 父节点熵：$H = -\left(\frac{7}{10}\log_2\frac{7}{10} + \frac{3}{10}\log_2\frac{3}{10}\right) \approx 0.881$

示例计算

- 父节点熵：$H = -\left(\frac{7}{10}\log_2\frac{7}{10} + \frac{3}{10}\log_2\frac{3}{10}\right) \approx 0.881$
- 子节点 1 熵：0.0 (纯节点)，子节点 2 熵：1.0 (不纯)

示例计算

- 父节点熵：$H = -\left(\frac{7}{10}\log_2\frac{7}{10} + \frac{3}{10}\log_2\frac{3}{10}\right) \approx 0.881$
- 子节点 1 熵：0.0 (纯节点)，子节点 2 熵：1.0 (不纯)
- 加权子节点熵：$\frac{3}{10} \times 0 + \frac{7}{10} \times 1 = 0.7$

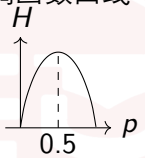
示例计算

- 父节点熵：$H = -\left(\frac{7}{10}\log_2\frac{7}{10} + \frac{3}{10}\log_2\frac{3}{10}\right) \approx 0.881$
- 子节点 1 熵：0.0 (纯节点)，子节点 2 熵：1.0 (不纯)
- 加权子节点熵：$\frac{3}{10} \times 0 + \frac{7}{10} \times 1 = 0.7$
- 信息增益：$0.881 - 0.7 = 0.181$

信息增益特性与变体

- **特性**:
  - 基于信息论，理论完备
  - ID3 算法使用
  - 偏向具有更多类别的特征
- **优点**:
  - 对类别不平衡较鲁棒
  - 产生更深的树
  - 可解释性强
- **缺点**:
  - 计算开销大（含对数运算）
  - 对连续特征敏感

熵函数曲线



信息增益率

为解决偏向多值特征问题，C4.5 算法使用信息增益率:

$$IGR = \frac{IG}{H_{\text{feature}}},$$

其中，H_{feature} 是特征本身的熵

基尼不纯度：CART 算法基础

- **定义**：衡量从节点中随机抽取两个样本，它们属于不同类别的概率。

基尼不纯度：CART 算法基础

- **定义**：衡量从节点中随机抽取两个样本，它们属于不同类别的概率。
- 另一种解释：从一个数据集中随机选取子项，度量其被错误的划分到其他组里的概率。

基尼不纯度：CART 算法基础

- **定义**：衡量从节点中随机抽取两个样本，它们属于不同类别的概率。
- 另一种解释：从一个数据集中随机选取子项，度量其被错误的划分到其他组里的概率。
- 计算公式：

$$G = 1 - \sum_{k=1}^{K} p_k^2,$$

其中，p_k 是节点中第 k 类样本的比例。

基尼不纯度：CART 算法基础

- **定义**：衡量从节点中随机抽取两个样本，它们属于不同类别的概率。
- 另一种解释：从一个数据集中随机选取子项，度量其被错误的划分到其他组里的概率。
- **计算公式**：

$$G = 1 - \sum_{k=1}^{K} p_k^2,$$

其中，p_k 是节点中第 k 类样本的比例。

- 值域：[0, 0.5]

基尼不纯度：CART 算法基础

- **定义**：衡量从节点中随机抽取两个样本，它们属于不同类别的概率。
- 另一种解释：从一个数据集中随机选取子项，度量其被错误的划分到其他组里的概率。
- **计算公式**：

$$G = 1 - \sum_{k=1}^{K} p_k^2,$$

其中，p_k 是节点中第 k 类样本的比例。

- **值域**：[0, 0.5]
  - 0：所有样本属于同一类别。

基尼不纯度：CART 算法基础

- **定义**：衡量从节点中随机抽取两个样本，它们属于不同类别的概率。
- 另一种解释：从一个数据集中随机选取子项，度量其被错误的划分到其他组里的概率。
- **计算公式**：

$$G = 1 - \sum_{k=1}^{K} p_k^2,$$

其中，p_k 是节点中第 k 类样本的比例。

- **值域**：[0, 0.5]
  - 0：所有样本属于同一类别。
  - 0.5：样本均匀分布在所有类别中。

基尼不纯度：CART 算法基础

- **定义**：衡量从节点中随机抽取两个样本，它们属于不同类别的概率。
- 另一种解释：从一个数据集中随机选取子项，度量其被错误的划分到其他组里的概率。
- **计算公式**：

$$G = 1 - \sum_{k=1}^{K} p_k^2,$$

其中，p_k 是节点中第 k 类样本的比例。

- **值域**：[0, 0.5]
  - 0：所有样本属于同一类别。
  - 0.5：样本均匀分布在所有类别中。
- **分裂准则**：选择使子节点基尼不纯度加权和最小的特征和分裂点。

$$\text{Gini}_{\text{split}} = \sum_{i=1}^{m} \frac{n_i}{n} \text{Gini}_i$$

基尼不纯度

示例计算

假设节点有 10 个样本：7 个 A 类，3 个 B 类

$$G = 1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2 = 1 - 0.49 - 0.09 = 0.42$$

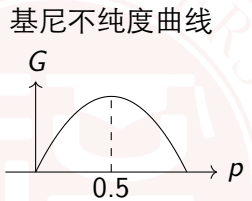
基尼不纯度特性与几何解释

- **特性**:

基尼不纯度曲线

基尼不纯度特性与几何解释

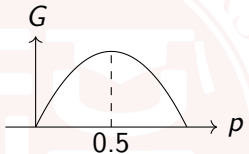
- **特性**:
  - 对类别分布变化敏感



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）

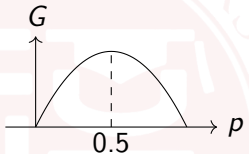
基尼不纯度曲线



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用

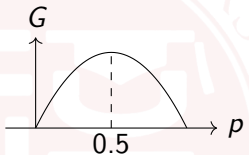
基尼不纯度曲线



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:

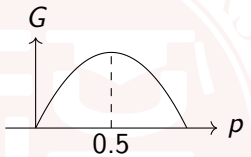
基尼不纯度曲线



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征

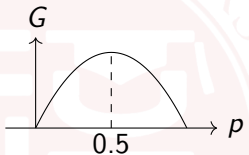
基尼不纯度曲线



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征
  - 计算高效

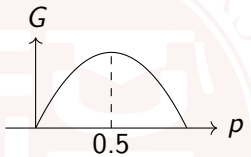
基尼不纯度曲线



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征
  - 计算高效
  - 产生平衡树

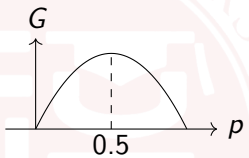
基尼不纯度曲线



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征
  - 计算高效
  - 产生平衡树
- **缺点**:

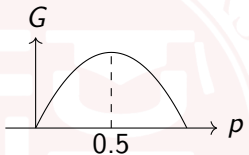
基尼不纯度曲线



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征
  - 计算高效
  - 产生平衡树
- **缺点**:
  - 对类别不平衡敏感

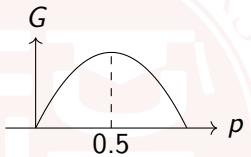
基尼不纯度曲线



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征
  - 计算高效
  - 产生平衡树
- **缺点**:
  - 对类别不平衡敏感
  - 可能产生相似子树

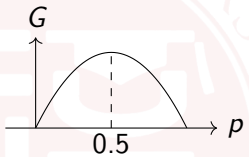
基尼不纯度曲线



基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征
  - 计算高效
  - 产生平衡树
- **缺点**:
  - 对类别不平衡敏感
  - 可能产生相似子树

基尼不纯度曲线

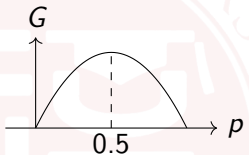


应用场景

基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征
  - 计算高效
  - 产生平衡树
- **缺点**:
  - 对类别不平衡敏感
  - 可能产生相似子树

基尼不纯度曲线



应用场景

- 二分类问题

基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征
  - 计算高效
  - 产生平衡树
- **缺点**:
  - 对类别不平衡敏感
  - 可能产生相似子树

基尼不纯度曲线

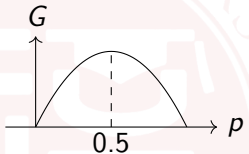
应用场景

- 二分类问题
- 类别较少的问题

基尼不纯度特性与几何解释

- **特性**:
  - 对类别分布变化敏感
  - 计算效率高（无对数运算）
  - CART 算法默认使用
- **优点**:
  - 偏向多值特征
  - 计算高效
  - 产生平衡树
- **缺点**:
  - 对类别不平衡敏感
  - 可能产生相似子树

基尼不纯度曲线

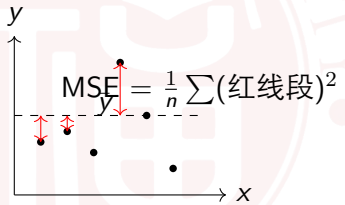


应用场景

- 二分类问题
- 类别较少的问题
- 计算资源受限场景

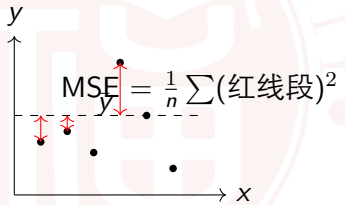
均方误差特性与优化

- **特性**:



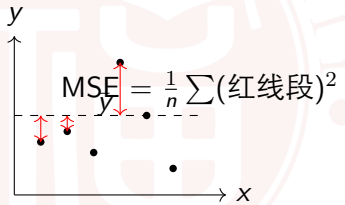
均方误差特性与优化

- **特性**:
  - 对异常值敏感（平方项放大影响）



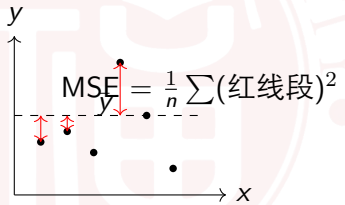
均方误差特性与优化

- **特性**：
  - 对异常值敏感（平方项放大影响）
  - 可导，便于优化



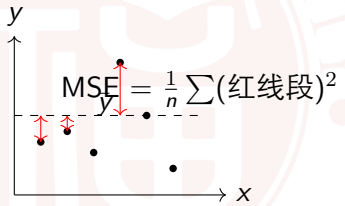
均方误差特性与优化

- **特性**:
  - 对异常值敏感（平方项放大影响）
  - 可导，便于优化
  - CART 回归树使用



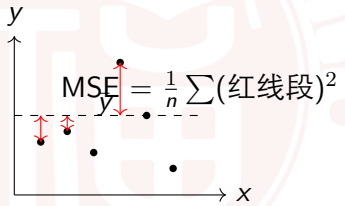
均方误差特性与优化

- **特性**:
  - 对异常值敏感（平方项放大影响）
  - 可导，便于优化
  - CART 回归树使用
- **优点**:



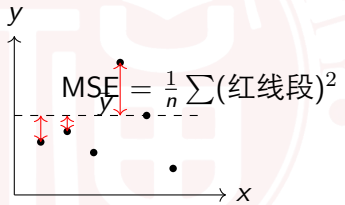
均方误差特性与优化

- **特性**:
  - 对异常值敏感（平方项放大影响）
  - 可导，便于优化
  - CART 回归树使用
- **优点**:
  - 数学性质良好



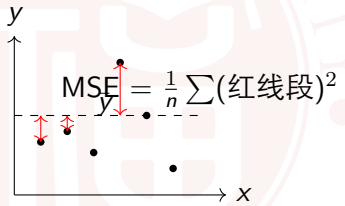
均方误差特性与优化

- **特性**:
  - 对异常值敏感（平方项放大影响）
  - 可导，便于优化
  - CART 回归树使用
- **优点**:
  - 数学性质良好
  - 预测值为节点均值，计算简单



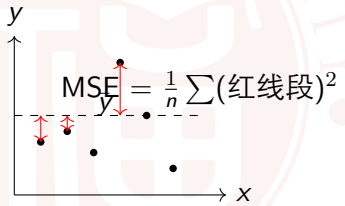
均方误差特性与优化

- **特性**:
  - 对异常值敏感（平方项放大影响）
  - 可导，便于优化
  - CART 回归树使用
- **优点**:
  - 数学性质良好
  - 预测值为节点均值，计算简单
  - 与许多回归模型目标一致



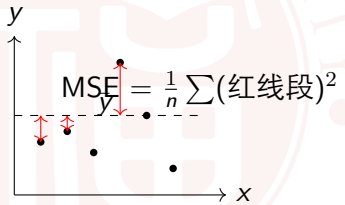
均方误差特性与优化

- **特性**:
  - 对异常值敏感（平方项放大影响）
  - 可导，便于优化
  - CART 回归树使用
- **优点**:
  - 数学性质良好
  - 预测值为节点均值，计算简单
  - 与许多回归模型目标一致
- **缺点**:



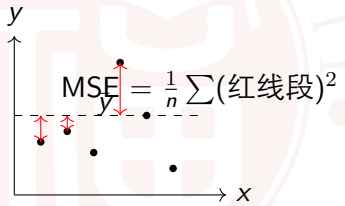
均方误差特性与优化

- **特性**:
  - 对异常值敏感（平方项放大影响）
  - 可导，便于优化
  - CART 回归树使用
- **优点**:
  - 数学性质良好
  - 预测值为节点均值，计算简单
  - 与许多回归模型目标一致
- **缺点**:
  - 对异常值敏感



均方误差特性与优化

- **特性**:
  - 对异常值敏感（平方项放大影响）
  - 可导，便于优化
  - CART 回归树使用
- **优点**:
  - 数学性质良好
  - 预测值为节点均值，计算简单
  - 与许多回归模型目标一致
- **缺点**:
  - 对异常值敏感
  - 可能产生偏差预测



三大准则对比分析

特性	基尼不纯度	信息增益	均方误差
适用问题	分类	分类	回归
计算效率	高	中（含对数）	高
对异常值敏感	低	低	高
类别不平衡鲁棒性	中	高	不适用
多值特征偏向	是	是	不适用
主流算法	CART	ID3/C4.5	CART 回归
输出预测	多数类	多数类	平均值

三大准则选择建议

- **分类问题**：基尼不纯度（效率高）或信息增益（理论完备）

三大准则选择建议

- **分类问题**：基尼不纯度（效率高）或信息增益（理论完备）
- **回归问题**：均方误差（标准方法）或 MAE（异常值多时）

三大准则选择建议

- **分类问题**：基尼不纯度（效率高）或信息增益（理论完备）
- **回归问题**：均方误差（标准方法）或 MAE（异常值多时）
- **实践考虑**：计算资源、数据特性、可解释性需求

实际应用与最佳实践

- **特征选择：**

实际应用与最佳实践

- **特征选择：**
  - 分类特征：使用基尼或信息增益

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：
  - 设置最小分裂增益阈值

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：
  - 设置最小分裂增益阈值
  - 限制最大树深度

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：
  - 设置最小分裂增益阈值
  - 限制最大树深度
  - 设置叶子节点最小样本数

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：
  - 设置最小分裂增益阈值
  - 限制最大树深度
  - 设置叶子节点最小样本数
- **处理过拟合**：

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：
  - 设置最小分裂增益阈值
  - 限制最大树深度
  - 设置叶子节点最小样本数
- **处理过拟合**：
  - 后剪枝（代价复杂度剪枝）

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：
  - 设置最小分裂增益阈值
  - 限制最大树深度
  - 设置叶子节点最小样本数
- **处理过拟合**：
  - 后剪枝（代价复杂度剪枝）
  - 集成方法（随机森林、GBDT）

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：
  - 设置最小分裂增益阈值
  - 限制最大树深度
  - 设置叶子节点最小样本数
- **处理过拟合**：
  - 后剪枝（代价复杂度剪枝）
  - 集成方法（随机森林、GBDT）
- **类别不平衡处理**：

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：
  - 设置最小分裂增益阈值
  - 限制最大树深度
  - 设置叶子节点最小样本数
- **处理过拟合**：
  - 后剪枝（代价复杂度剪枝）
  - 集成方法（随机森林、GBDT）
- **类别不平衡处理**：
  - 类权重调整

实际应用与最佳实践

- **特征选择**：
  - 分类特征：使用基尼或信息增益
  - 连续特征：需要寻找最佳分裂点（排序后遍历）
- **预剪枝策略**：
  - 设置最小分裂增益阈值
  - 限制最大树深度
  - 设置叶子节点最小样本数
- **处理过拟合**：
  - 后剪枝（代价复杂度剪枝）
  - 集成方法（随机森林、GBDT）
- **类别不平衡处理**：
  - 类权重调整
  - 使用信息增益或平衡基尼指数

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
  - 决策树
  - 集成学习与随机森林
  - AdaBoost
  - Gradient Boosting
  - XGBoost
  - LightGBM
  - 核方法

集成学习基本概念

- 核心思想：组合多个弱学习器构建强学习器

集成学习优势

集成学习基本概念

- **核心思想**：组合多个弱学习器构建强学习器
- **哲学原理**：“三个臭皮匠，顶个诸葛亮”

集成学习优势

集成学习基本概念

- **核心思想**：组合多个弱学习器构建强学习器
- **哲学原理**：“三个臭皮匠，顶个诸葛亮”
- **关键要素**：

集成学习优势

集成学习基本概念

- **核心思想**：组合多个弱学习器构建强学习器
- **哲学原理**：“三个臭皮匠，顶个诸葛亮”
- **关键要素**：
  - 个体学习器应有一定的“准确性”

集成学习优势

集成学习基本概念

- **核心思想**：组合多个弱学习器构建强学习器
- **哲学原理**：“三个臭皮匠，顶个诸葛亮”
- **关键要素**：
  - 个体学习器应有一定的“准确性”
  - 学习器之间需要“多样性”

集成学习优势

集成学习基本概念

- **核心思想**：组合多个弱学习器构建强学习器
- **哲学原理**：“三个臭皮匠，顶个诸葛亮”
- **关键要素**：
  - 个体学习器应有一定的“准确性”
  - 学习器之间需要“多样性”

集成学习优势

- 提升预测准确性和稳定性

集成学习基本概念

- **核心思想**：组合多个弱学习器构建强学习器
- **哲学原理**：“三个臭皮匠，顶个诸葛亮”
- **关键要素**：
  - 个体学习器应有一定的“准确性”
  - 学习器之间需要“多样性”

集成学习优势

- 提升预测准确性和稳定性
- 降低过拟合风险，增强鲁棒性

集成学习基本概念

- **核心思想**：组合多个弱学习器构建强学习器
- **哲学原理**：“三个臭皮匠，顶个诸葛亮”
- **关键要素**：
  - 个体学习器应有一定的“准确性”
  - 学习器之间需要“多样性”

集成学习优势

- 提升预测准确性和稳定性
- 降低过拟合风险，增强鲁棒性
- 适用于分类、回归等多种任务

集成学习的理论基础

方差-偏差分解

$$\text{误差} = \text{偏差}^2 + \text{方差} + \text{噪声}$$

- **降低方差**：通过平均多个不相关的树预测值
- **独立性假设**：若基学习器相互独立，集成后方差降为原来的 $1/T$

$$\text{Var}\left(\frac{1}{T}\sum_{t=1}^{T} h_t(x)\right) = \frac{1}{T^2}\sum_{t=1}^{T}\text{Var}(h_t(x)) = \frac{\sigma^2}{T}$$

Bagging 算法起源

- **提出者**：Leo Breiman（1996 年）
- **全称**：Bootstrap Aggregating（自助聚合）
- **基本思想**：通过 Bootstrap 采样构建多个模型并集成

$$\text{Bagging} = \text{Bootstrap 数据采样} + \text{模型聚合}$$

Bagging 算法流程

1. **自助采样**：生成 T 个 Bootstrap 样本子集

Bagging 算法流程

1. **自助采样**：生成 T 个 Bootstrap 样本子集
2. **基学习器训练**：每个子集训练一个基学习器

Bagging 算法流程

1. **自助采样**：生成 T 个 Bootstrap 样本子集
2. **基学习器训练**：每个子集训练一个基学习器
3. **结果聚合**：

Bagging 算法流程

1. **自助采样**：生成 T 个 Bootstrap 样本子集
2. **基学习器训练**：每个子集训练一个基学习器
3. **结果聚合**：
   - 分类任务：多数投票法

Bagging 算法流程

1. **自助采样**：生成 T 个 Bootstrap 样本子集
2. **基学习器训练**：每个子集训练一个基学习器
3. **结果聚合**：
   - 分类任务：多数投票法
   - 回归任务：简单平均法

Bagging 算法流程

1. **自助采样**：生成 T 个 Bootstrap 样本子集
2. **基学习器训练**：每个子集训练一个基学习器
3. **结果聚合**：
   - 分类任务：多数投票法
   - 回归任务：简单平均法

Bagging 算法流程

1. **自助采样**：生成 T 个 Bootstrap 样本子集
2. **基学习器训练**：每个子集训练一个基学习器
3. **结果聚合**：
   - 分类任务：多数投票法
   - 回归任务：简单平均法

$$\text{分类：}\ H(x) = \arg\max_{y \in \mathcal{Y}} \sum_{t=1}^{T} \mathbb{I}(h_t(x) = y)$$

Bagging 算法流程

1. **自助采样**：生成 T 个 Bootstrap 样本子集
2. **基学习器训练**：每个子集训练一个基学习器
3. **结果聚合**：
   - 分类任务：多数投票法
   - 回归任务：简单平均法

$$\text{分类：}\ H(x) = \arg\max_{y \in \mathcal{Y}} \sum_{t=1}^{T} \mathbb{I}(h_t(x) = y)$$

$$\text{回归：}\ H(x) = \frac{1}{T} \sum_{t=1}^{T} h_t(x)$$

Bagging 算法优缺点

优点

- **降低方差**：特别适合高方差模型

Bagging 算法优缺点

优点

- **降低方差**：特别适合高方差模型
- **并行训练**：基学习器相互独立

Bagging 算法优缺点

优点

- **降低方差**：特别适合高方差模型
- **并行训练**：基学习器相互独立
- **稳定性强**：对异常值不敏感

Bagging 算法优缺点

优点

- **降低方差**：特别适合高方差模型
- **并行训练**：基学习器相互独立
- **稳定性强**：对异常值不敏感
- **OOB 估计**：无需额外验证集

Bagging 算法优缺点

优点

- **降低方差**：特别适合高方差模型
- **并行训练**：基学习器相互独立
- **稳定性强**：对异常值不敏感
- **OOB 估计**：无需额外验证集

缺点

Bagging 算法优缺点

优点

- **降低方差**：特别适合高方差模型
- **并行训练**：基学习器相互独立
- **稳定性强**：对异常值不敏感
- **OOB 估计**：无需额外验证集

缺点

- **计算成本高**：需训练多个模型

Bagging 算法优缺点

优点

- **降低方差**：特别适合高方差模型
- **并行训练**：基学习器相互独立
- **稳定性强**：对异常值不敏感
- **OOB 估计**：无需额外验证集

缺点

- **计算成本高**：需训练多个模型
- **可解释性差**：模型结构复杂

Bagging 算法优缺点

优点

- **降低方差**：特别适合高方差模型
- **并行训练**：基学习器相互独立
- **稳定性强**：对异常值不敏感
- **OOB 估计**：无需额外验证集

缺点

- **计算成本高**：需训练多个模型
- **可解释性差**：模型结构复杂
- **降偏差有限**：对高偏差模型改进有限

随机森林算法介绍

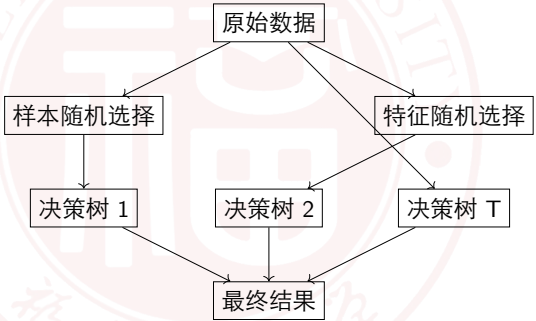
- **提出者**：Leo Breiman（2001 年）
- **定义**：Bagging 的扩展，以决策树为基学习器
- **核心创新**：在 Bagging 基础上引入**特征随机性**

随机森林 vs 普通 Bagging

随机森林在样本随机性的基础上增加了特征随机选择，进一步增强了模型的多样性和泛化能力

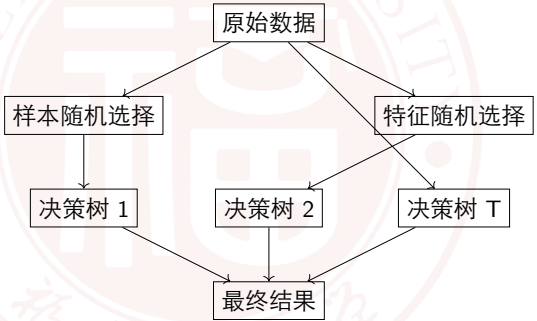
随机森林的双重随机性

1. **数据随机性**



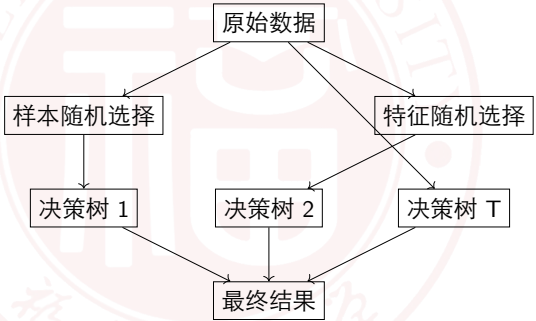
随机森林的双重随机性

1. **数据随机性**
   - Bootstrap 采样生成训练子集



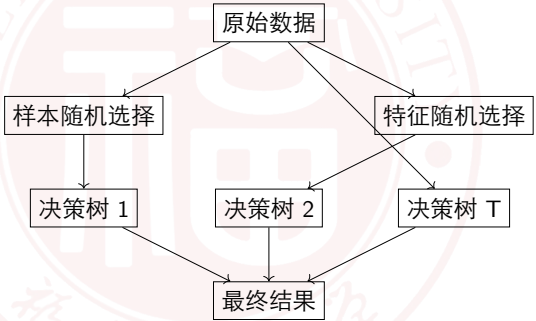
随机森林的双重随机性

1. **数据随机性**
   - Bootstrap 采样生成训练子集
   - 每个决策树使用不同的训练数据



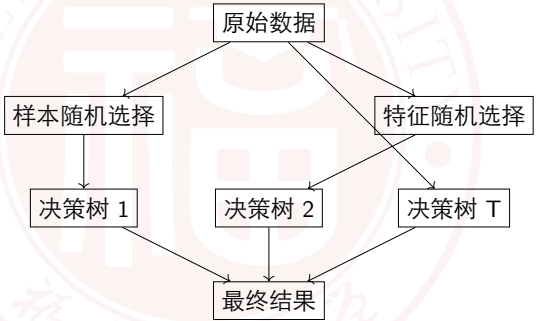
随机森林的双重随机性

1. **数据随机性**
   - Bootstrap 采样生成训练子集
   - 每个决策树使用不同的训练数据
2. **特征随机性**



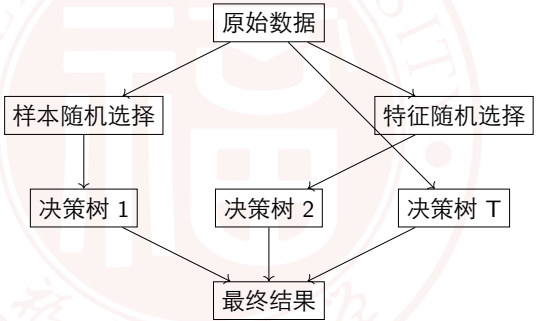
随机森林的双重随机性

1. **数据随机性**
   - Bootstrap 采样生成训练子集
   - 每个决策树使用不同的训练数据
2. **特征随机性**
   - 节点分裂时随机选择特征子集



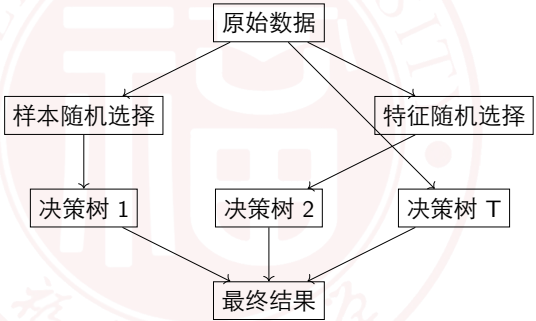
随机森林的双重随机性

1. **数据随机性**
   - Bootstrap 采样生成训练子集
   - 每个决策树使用不同的训练数据
2. **特征随机性**
   - 节点分裂时随机选择特征子集
   - 传统决策树：考虑所有特征



随机森林的双重随机性

1. **数据随机性**
   - Bootstrap 采样生成训练子集
   - 每个决策树使用不同的训练数据
2. **特征随机性**
   - 节点分裂时随机选择特征子集
   - 传统决策树：考虑所有特征
   - 随机森林：考虑 k 个随机特征 ($k \ll d$)



随机森林构建过程

1. **数据准备**：从原始数据集中有放回抽样生成多个训练子集

分类：$H(x) = \text{mode}\{h_t(x)\}_{t=1}^{T}$

回归：$H(x) = \frac{1}{T}\sum_{t=1}^{T} h_t(x)$

随机森林构建过程

1. **数据准备**：从原始数据集中有放回抽样生成多个训练子集
2. **树构建**：对每个训练子集构建决策树，节点分裂时随机选择特征子集

分类：$H(x) = \text{mode}\{h_t(x)\}_{t=1}^{T}$

回归：$H(x) = \frac{1}{T}\sum_{t=1}^{T} h_t(x)$

随机森林构建过程

1. **数据准备**：从原始数据集中有放回抽样生成多个训练子集
2. **树构建**：对每个训练子集构建决策树，节点分裂时随机选择特征子集
3. **生长策略**：决策树完全生长，不进行剪枝

$$\text{分类：} H(x) = \text{mode}\{h_t(x)\}_{t=1}^{T}$$

$$\text{回归：} H(x) = \frac{1}{T}\sum_{t=1}^{T} h_t(x)$$

随机森林构建过程

1. **数据准备**：从原始数据集中有放回抽样生成多个训练子集
2. **树构建**：对每个训练子集构建决策树，节点分裂时随机选择特征子集
3. **生长策略**：决策树完全生长，不进行剪枝
4. **结果集成**：多棵决策树通过投票（分类）或平均（回归）组合

$$\text{分类：} H(x) = \text{mode}\{h_t(x)\}_{t=1}^{T}$$

$$\text{回归：} H(x) = \frac{1}{T}\sum_{t=1}^{T} h_t(x)$$

随机森林关键参数

随机森林主要参数及设置建议

参数	推荐值	说明
n_estimators	100-500	树的数量，越多越好但计算成本增加
max_features	$\sqrt{d}$	特征子集大小，分类常用 $\sqrt{d}$，回归常用 $d/3$
max_depth	None	树的最大深度，控制过拟合
min_samples_split	2	内部节点分裂所需最小样本数
min_samples_leaf	1	叶节点最少样本数

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练
- **特征重要性**：可评估特征对预测的贡献度

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练
- **特征重要性**：可评估特征对预测的贡献度
- **处理缺失值**：对缺失数据有较好鲁棒性

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练
- **特征重要性**：可评估特征对预测的贡献度
- **处理缺失值**：对缺失数据有较好鲁棒性
- **无需特征缩放**：不受特征尺度影响

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练
- **特征重要性**：可评估特征对预测的贡献度
- **处理缺失值**：对缺失数据有较好鲁棒性
- **无需特征缩放**：不受特征尺度影响

局限

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练
- **特征重要性**：可评估特征对预测的贡献度
- **处理缺失值**：对缺失数据有较好鲁棒性
- **无需特征缩放**：不受特征尺度影响

局限

- **计算复杂度**：树数量多时训练和预测较慢

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练
- **特征重要性**：可评估特征对预测的贡献度
- **处理缺失值**：对缺失数据有较好鲁棒性
- **无需特征缩放**：不受特征尺度影响

局限

- **计算复杂度**：树数量多时训练和预测较慢
- **内存消耗**：需要存储多棵决策树

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练
- **特征重要性**：可评估特征对预测的贡献度
- **处理缺失值**：对缺失数据有较好鲁棒性
- **无需特征缩放**：不受特征尺度影响

局限

- **计算复杂度**：树数量多时训练和预测较慢
- **内存消耗**：需要存储多棵决策树
- **可解释性差**：相比单棵决策树更难解释

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练
- **特征重要性**：可评估特征对预测的贡献度
- **处理缺失值**：对缺失数据有较好鲁棒性
- **无需特征缩放**：不受特征尺度影响

局限

- **计算复杂度**：树数量多时训练和预测较慢
- **内存消耗**：需要存储多棵决策树
- **可解释性差**：相比单棵决策树更难解释
- **小数据集表现**：数据量太少时效果不佳

随机森林的优劣

优势

- **高准确性**：集成多棵树，降低过拟合风险
- **鲁棒性强**：对噪声数据和异常值不敏感
- **并行化**：树之间独立，可并行训练
- **特征重要性**：可评估特征对预测的贡献度
- **处理缺失值**：对缺失数据有较好鲁棒性
- **无需特征缩放**：不受特征尺度影响

局限

- **计算复杂度**：树数量多时训练和预测较慢
- **内存消耗**：需要存储多棵决策树
- **可解释性差**：相比单棵决策树更难解释
- **小数据集表现**：数据量太少时效果不佳
- **外推能力**：对超出训练范围的数据预测能力有限

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
  - 决策树
  - 集成学习与随机森林
  - AdaBoost
  - Gradient Boosting
  - XGBoost
  - LightGBM
  - 核方法

Boosting 算法发展历程

- **1990 年**：Schapire 提出首个 Boosting 算法理论框架

Boosting 算法发展历程

- **1990 年**：Schapire 提出首个 Boosting 算法理论框架
- **1995 年**：Freund 和 Schapire 提出 AdaBoost 算法

Boosting 算法发展历程

- **1990 年**：Schapire 提出首个 Boosting 算法理论框架
- **1995 年**：Freund 和 Schapire 提出 AdaBoost 算法
- **2000 年代**：多种改进算法涌现，解决实际应用问题

Boosting 算法发展历程

- **1990 年**：Schapire 提出首个 Boosting 算法理论框架
- **1995 年**：Freund 和 Schapire 提出 AdaBoost 算法
- **2000 年代**：多种改进算法涌现，解决实际应用问题
- **当前**：成为集成学习中的重要基础算法

Boosting 算法发展历程

- **1990 年**：Schapire 提出首个 Boosting 算法理论框架
- **1995 年**：Freund 和 Schapire 提出 AdaBoost 算法
- **2000 年代**：多种改进算法涌现，解决实际应用问题
- **当前**：成为集成学习中的重要基础算法

时间	里程碑	核心贡献
1990 年	Schapire 的 Boosting	理论证明弱学习器可提升为强学习器
1995 年	AdaBoost	自适应权重调整机制
2000 年代	多种改进算法	解决过拟合、噪声敏感等问题

Schapire 的 Boosting

- **核心问题**：弱学习器能否提升为强学习器?

Schapire 的 Boosting

- **核心问题**：弱学习器能否提升为强学习器?
- **理论基础**：在 PAC（Probably Approximately Correct）学习框架下，弱学习与强学习等价。

Schapire 的 Boosting

- **核心问题**：弱学习器能否提升为强学习器?
- **理论基础**：在 PAC（Probably Approximately Correct）学习框架下，弱学习与强学习等价。

PAC 理论

一个概念类 C 被称为是 PAC 可学习的，如果存在一个学习算法 A 和一个多项式函数 $\mathrm{poly}(\,)$，使得对于任意分布 D、任意目标概念 $c \in C$、以及任意 $0 < \varepsilon, \delta < 1$，只要提供给算法 A 的样本数量 m 满足：

$$m \geq \mathrm{poly}(\frac{1}{\varepsilon}, \frac{1}{\delta}, n, \mathrm{size}(c))$$

那么算法 A 就能在多项式时间内，以至少 $1-\delta$ 的概率输出一个假设 h，其错误率 $error_D(h) \leq \varepsilon$。

Schapire 的 Boosting

- **核心问题**：弱学习器能否提升为强学习器?
- **理论基础**：在 PAC（Probably Approximately Correct）学习框架下，弱学习与强学习等价。
- 三层级联架构:

Schapire 的 Boosting

- **核心问题**：弱学习器能否提升为强学习器?
- **理论基础**：在 PAC（Probably Approximately Correct）学习框架下，弱学习与强学习等价。
- **三层级联架构**：
  1. 第一层：在原始数据上训练分类器。

Schapire 的 Boosting

- **核心问题**：弱学习器能否提升为强学习器?
- **理论基础**：在 PAC（Probably Approximately Correct）学习框架下，弱学习与强学习等价。
- **三层级联架构**：
  1. 第一层：在原始数据上训练分类器。
  2. 第二层：在第一个分类器错分样本上训练。

Schapire 的 Boosting

- **核心问题**：弱学习器能否提升为强学习器?
- **理论基础**：在 PAC（Probably Approximately Correct）学习框架下，弱学习与强学习等价。
- **三层级联架构**：
  1. 第一层：在原始数据上训练分类器。
  2. 第二层：在第一个分类器错分样本上训练。
  3. 第三层：在前两个分类器分歧样本上训练。
- **局限性**：需要预先知道弱学习器正确率下限。

Schapire 的 Boosting

- **核心问题**：弱学习器能否提升为强学习器?
- **理论基础**：在 PAC（Probably Approximately Correct）学习框架下，弱学习与强学习等价。
- **三层级联架构**：
  1. 第一层：在原始数据上训练分类器。
  2. 第二层：在第一个分类器错分样本上训练。
  3. 第三层：在前两个分类器分歧样本上训练。
- **局限性**：需要预先知道弱学习器正确率下限。

理论意义

首次从理论上证明了“弱可学习”与“强可学习”的等价性，为后续 Boosting 算法奠定基础。

AdaBoost

- **核心创新**：自适应样本权重调整机制

AdaBoost

- **核心创新**：自适应样本权重调整机制
- **关键改进**：无需预先知道弱学习器正确率下限

AdaBoost

- **核心创新**：自适应样本权重调整机制
- **关键改进**：无需预先知道弱学习器正确率下限

算法核心步骤

AdaBoost

- **核心创新**：自适应样本权重调整机制
- **关键改进**：无需预先知道弱学习器正确率下限

算法核心步骤

1. 初始化样本权重

AdaBoost

- **核心创新**：自适应样本权重调整机制
- **关键改进**：无需预先知道弱学习器正确率下限

算法核心步骤

1. 初始化样本权重
2. 迭代训练弱分类器

AdaBoost

- **核心创新**：自适应样本权重调整机制
- **关键改进**：无需预先知道弱学习器正确率下限

算法核心步骤

1. 初始化样本权重
2. 迭代训练弱分类器
3. 自适应调整权重

$$D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

- D 样本权重
- α 分类器权重
- Z_t 归一化权重 $\sum_i D_{t+1}(i)$

AdaBoost

- **核心创新**：自适应样本权重调整机制
- **关键改进**：无需预先知道弱学习器正确率下限

算法核心步骤

1. 初始化样本权重
2. 迭代训练弱分类器
3. 自适应调整权重

$$D_{t+1}(i) = \frac{D_t(i)\exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

4. 加权组合弱分类器

$$\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

AdaBoost 总结

改进与发展

- **权重更新方法改进:**

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：
  - FloatBoost：替换表现差的弱分类器

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：
  - FloatBoost：替换表现差的弱分类器
  - SEAdaBoost：结合 AdaBoost 和 SVM

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：
  - FloatBoost：替换表现差的弱分类器
  - SEAdaBoost：结合 AdaBoost 和 SVM
- **综合改进**：

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：
  - FloatBoost：替换表现差的弱分类器
  - SEAdaBoost：结合 AdaBoost 和 SVM
- **综合改进**：
  - EMV-AdaBoost：每轮训练两个弱分类器

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：
  - FloatBoost：替换表现差的弱分类器
  - SEAdaBoost：结合 AdaBoost 和 SVM
- **综合改进**：
  - EMV-AdaBoost：每轮训练两个弱分类器
  - EAdaBoost：结合 kNN 处理噪声

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：
  - FloatBoost：替换表现差的弱分类器
  - SEAdaBoost：结合 AdaBoost 和 SVM
- **综合改进**：
  - EMV-AdaBoost：每轮训练两个弱分类器
  - EAdaBoost：结合 kNN 处理噪声

优势与局限

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：
  - FloatBoost：替换表现差的弱分类器
  - SEAdaBoost：结合 AdaBoost 和 SVM
- **综合改进**：
  - EMV-AdaBoost：每轮训练两个弱分类器
  - EAdaBoost：结合 kNN 处理噪声

优势与局限

- **优势**：简单有效，无需预设弱学习器正确率下限。

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：
  - FloatBoost：替换表现差的弱分类器
  - SEAdaBoost：结合 AdaBoost 和 SVM
- **综合改进**：
  - EMV-AdaBoost：每轮训练两个弱分类器
  - EAdaBoost：结合 kNN 处理噪声

优势与局限

- **优势**：简单有效，无需预设弱学习器正确率下限。
- **局限**：仅支持特定损失函数（如指数损失），难以扩展至回归问题。

AdaBoost 总结

改进与发展

- **权重更新方法改进**：
  - AsymBoost：处理不平衡数据
  - RADA 算法：抑制误分类样本权重
  - NAdaBoost：设置困难样本权重上限
- **训练方法改进**：
  - FloatBoost：替换表现差的弱分类器
  - SEAdaBoost：结合 AdaBoost 和 SVM
- **综合改进**：
  - EMV-AdaBoost：每轮训练两个弱分类器
  - EAdaBoost：结合 kNN 处理噪声

优势与局限

- **优势**：简单有效，无需预设弱学习器正确率下限。
- **局限**：仅支持特定损失函数（如指数损失），难以扩展至回归问题。

实际应用影响

AdaBoost 及其改进算法在人脸检测、数据挖掘等领域取得显著成功，推动了集成学习的发展

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
  - 决策树
  - 集成学习与随机森林
  - AdaBoost
  - Gradient Boosting
  - XGBoost
  - LightGBM
  - 核方法

从权重调整到梯度优化

- **核心问题**：AdaBoost 的权重调整机制缺乏一般性，无法灵活适应回归、多分类等任务。

从权重调整到梯度优化

- **核心问题**：AdaBoost 的权重调整机制缺乏一般性，无法灵活适应回归、多分类等任务。
- **关键创新**：1999 年 Friedman 提出 Gradient Boosting，将 Boosting 框架转化为梯度下降优化问题。

从权重调整到梯度优化

- **核心问题**：AdaBoost 的权重调整机制缺乏一般性，无法灵活适应回归、多分类等任务。
- **关键创新**：1999 年 Friedman 提出 Gradient Boosting，将 Boosting 框架转化为梯度下降优化问题。

算法框架

$$F_m(x) = F_{m-1}(x) + \rho_m h_m(x)$$

其中 ρ_m 为步长（学习率），$h_m(x)$ 为本轮训练的基学习器。

从权重调整到梯度优化

- **核心问题**：AdaBoost 的权重调整机制缺乏一般性，无法灵活适应回归、多分类等任务。
- **关键创新**：1999 年 Friedman 提出 Gradient Boosting，将 Boosting 框架转化为梯度下降优化问题。

算法框架

$$F_m(x) = F_{m-1}(x) + \rho_m h_m(x)$$

其中 ρ_m 为步长（学习率），$h_m(x)$ 为本轮训练的基学习器。

- **梯度下降思想**：每轮训练一个弱学习器来拟合当前模型的负梯度方向（即残差近似值），使损失函数沿梯度下降最快方向减少。

从权重调整到梯度优化

- **核心问题**：AdaBoost 的权重调整机制缺乏一般性，无法灵活适应回归、多分类等任务。
- **关键创新**：1999 年 Friedman 提出 Gradient Boosting，将 Boosting 框架转化为梯度下降优化问题。

算法框架

$$F_m(x) = F_{m-1}(x) + \rho_m h_m(x)$$

其中 ρ_m 为步长（学习率），$h_m(x)$ 为本轮训练的基学习器。

- **梯度下降思想**：每轮训练一个弱学习器来拟合当前模型的负梯度方向（即残差近似值），使损失函数沿梯度下降最快方向减少。
- **泛化能力**：可适配任意可微损失函数（如均方误差、绝对损失、交叉熵等）。

GBRT 的数学原理

- **基学习器**：限定使用 CART 回归树，因回归树输出可加减，适合累加组合。

负梯度（伪残差）：$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$

GBRT 的数学原理

- **基学习器**：限定使用 CART 回归树，因回归树输出可加减，适合累加组合。
- **目标**：最小化损失函数 $L(y, F(x))$，通过加法模型逐步优化。

负梯度（伪残差）：$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$

GBRT 的数学原理

- **基学习器**：限定使用 CART 回归树，因回归树输出可加减，适合累加组合。
- **目标**：最小化损失函数 $L(y, F(x))$，通过加法模型逐步优化。

负梯度（伪残差）：$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$

直观案例：年龄预测

真实年龄 30 岁 $\rightarrow$ 第一棵树预测 20 岁（残差 10 岁）$\rightarrow$ 第二棵树预测 6 岁（残差 4 岁）$\rightarrow$ 第三棵树预测 3 岁（残差 1 岁）。最终预测：$20 + 6 + 3 = 29$ 岁（接近真实值）。

GBRT 的数学原理

- **基学习器**：限定使用 CART 回归树，因回归树输出可加减，适合累加组合。
- **目标**：最小化损失函数 $L(y, F(x))$，通过加法模型逐步优化。

负梯度（伪残差）：$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$

直观案例：年龄预测

真实年龄 30 岁 $\rightarrow$ 第一棵树预测 20 岁（残差 10 岁）$\rightarrow$ 第二棵树预测 6 岁（残差 4 岁）$\rightarrow$ 第三棵树预测 3 岁（残差 1 岁）。最终预测：$20 + 6 + 3 = 29$ 岁（接近真实值）。

- **关键点**：每棵树学习的是前序模型累积预测的残差，通过梯度下降近似残差拟合。

GBRT 算法详细流程

1. **初始化**：常数值预测，如 $F_0(x) = \arg\min_c \sum L(y_i, c)$。

GBRT 算法详细流程

1. **初始化**：常数值预测，如 $F_0(x) = \arg\min_c \sum L(y_i, c)$。
2. **迭代训练**（对于每轮 $m = 1$ 到 M）：

GBRT 算法详细流程

1. **初始化**：常数值预测，如 $F_0(x) = \arg\min_c \sum L(y_i, c)$。
2. **迭代训练**（对于每轮 $m = 1$ 到 M）：
   - 计算伪残差：$r_{mi} = -\partial L / \partial F|_{F=F_{m-1}}$

GBRT 算法详细流程

1. **初始化**：常数值预测，如 $F_0(x) = \arg\min_c \sum L(y_i, c)$。
2. **迭代训练**（对于每轮 $m = 1$ 到 M）：
   - 计算伪残差：$r_{mi} = -\partial L / \partial F|_{F=F_{m-1}}$
   - 用 (x_i, r_{mi}) 训练回归树 $h_m(x)$，生成叶节点区域 R_{mj}

GBRT 算法详细流程

1. **初始化**：常数值预测，如 $F_0(x) = \arg\min_c \sum L(y_i, c)$。
2. **迭代训练**（对于每轮 $m = 1$ 到 M）：
   - 计算伪残差：$r_{mi} = -\partial L / \partial F|_{F=F_{m-1}}$
   - 用 (x_i, r_{mi}) 训练回归树 $h_m(x)$，生成叶节点区域 R_{mj}
   - 计算叶节点输出：$c_{mj} = \arg\min_c \sum_{x_i \in R_{mj}} L(y_i, F_{m-1}(x_i) + c)$

GBRT 算法详细流程

1. **初始化**：常数值预测，如 $F_0(x) = \arg\min_c \sum L(y_i, c)$。
2. **迭代训练**（对于每轮 $m = 1$ 到 M）：
   - 计算伪残差：$r_{mi} = -\partial L / \partial F|_{F=F_{m-1}}$
   - 用 (x_i, r_{mi}) 训练回归树 $h_m(x)$，生成叶节点区域 R_{mj}
   - 计算叶节点输出：$c_{mj} = \arg\min_c \sum_{x_i \in R_{mj}} L(y_i, F_{m-1}(x_i) + c)$
   - 更新模型：$F_m(x) = F_{m-1}(x) + \nu \cdot \sum_j c_{mj} I(x \in R_{mj})$（$\nu$ 为学习率）

GBRT 算法详细流程

1. **初始化**：常数值预测，如 $F_0(x) = \arg\min_c \sum L(y_i, c)$。
2. **迭代训练**（对于每轮 $m = 1$ 到 M）：
   - 计算伪残差：$r_{mi} = -\partial L / \partial F|_{F=F_{m-1}}$
   - 用 (x_i, r_{mi}) 训练回归树 $h_m(x)$，生成叶节点区域 R_{mj}
   - 计算叶节点输出：$c_{mj} = \arg\min_c \sum_{x_i \in R_{mj}} L(y_i, F_{m-1}(x_i) + c)$
   - 更新模型：$F_m(x) = F_{m-1}(x) + \nu \cdot \sum_j c_{mj} I(x \in R_{mj})$（$\nu$ 为学习率）
3. **最终模型**：$F_M(x) = F_0(x) + \sum_{m=1}^{M} \nu \cdot h_m(x)$。

GBRT 算法详细流程

1. **初始化**：常数值预测，如 $F_0(x) = \arg\min_c \sum L(y_i, c)$。
2. **迭代训练**（对于每轮 $m = 1$ 到 M）：
   - 计算伪残差：$r_{mi} = -\partial L / \partial F|_{F=F_{m-1}}$
   - 用 (x_i, r_{mi}) 训练回归树 $h_m(x)$，生成叶节点区域 R_{mj}
   - 计算叶节点输出：$c_{mj} = \arg\min_c \sum_{x_i \in R_{mj}} L(y_i, F_{m-1}(x_i) + c)$
   - 更新模型：$F_m(x) = F_{m-1}(x) + \nu \cdot \sum_j c_{mj} I(x \in R_{mj})$（$\nu$ 为学习率）
3. **最终模型**：$F_M(x) = F_0(x) + \sum_{m=1}^{M} \nu \cdot h_m(x)$。

学习率的作用

ν（通常 0.01–0.1）控制每棵树的贡献度，较小值需更多树但降低过拟合风险。

GBRT 正则化

- **过拟合风险**：复杂树结构或过多迭代可能导致模型过拟合。

GBRT 正则化

- **过拟合风险**：复杂树结构或过多迭代可能导致模型过拟合。
- **三大正则化手段**：

GBRT 正则化

- **过拟合风险**：复杂树结构或过多迭代可能导致模型过拟合。
- **三大正则化手段**：
  1. **学习率收缩**：减小每棵树的影响，需增加树数量平衡。

GBRT 正则化

- **过拟合风险**：复杂树结构或过多迭代可能导致模型过拟合。
- **三大正则化手段**：
  1. **学习率收缩**：减小每棵树的影响，需增加树数量平衡。
  2. **子树采样**：每轮随机选择部分样本或特征训练（如 subsample=0.8），类似随机森林的随机性注入。

GBRT 正则化

- **过拟合风险**：复杂树结构或过多迭代可能导致模型过拟合。
- **三大正则化手段**：
  1. **学习率收缩**：减小每棵树的影响，需增加树数量平衡。
  2. **子树采样**：每轮随机选择部分样本或特征训练（如 subsample=0.8），类似随机森林的随机性注入。
  3. **树复杂度控制**：限制树深度、叶节点最小样本数等。

GBRT 正则化

- **过拟合风险**：复杂树结构或过多迭代可能导致模型过拟合。
- **三大正则化手段**：
  1. **学习率收缩**：减小每棵树的影响，需增加树数量平衡。
  2. **子树采样**：每轮随机选择部分样本或特征训练（如 subsample=0.8），类似随机森林的随机性注入。
  3. **树复杂度控制**：限制树深度、叶节点最小样本数等。

正则化效果

通过约束模型复杂度，GBRT 在偏差和方差间取得平衡，提升泛化能力。

GBRT 正则化

- **过拟合风险**：复杂树结构或过多迭代可能导致模型过拟合。
- **三大正则化手段**：
  1. **学习率收缩**：减小每棵树的影响，需增加树数量平衡。
  2. **子树采样**：每轮随机选择部分样本或特征训练（如 subsample=0.8），类似随机森林的随机性注入。
  3. **树复杂度控制**：限制树深度、叶节点最小样本数等。

正则化效果

通过约束模型复杂度，GBRT 在偏差和方差间取得平衡，提升泛化能力。

- **调参关键**：树数量（n_estimators）、学习率（learning_rate）、树深度（max_depth）需交叉验证优化。

优势与局限性

优势

- **高精度**：处理复杂非线性关系。

优势与局限性

优势

- **高精度**：处理复杂非线性关系。
- **灵活性**：支持多种损失函数和数据类型。

优势与局限性

优势

- **高精度**：处理复杂非线性关系。
- **灵活性**：支持多种损失函数和数据类型。
- **鲁棒性**：对缺失值、异常值不敏感。

优势与局限性

优势

- **高精度**：处理复杂非线性关系。
- **灵活性**：支持多种损失函数和数据类型。
- **鲁棒性**：对缺失值、异常值不敏感。
- **可解释性**：提供特征重要性评估。

优势与局限性

优势

- **高精度**：处理复杂非线性关系。
- **灵活性**：支持多种损失函数和数据类型。
- **鲁棒性**：对缺失值、异常值不敏感。
- **可解释性**：提供特征重要性评估。

局限性

优势与局限性

优势

- **高精度**：处理复杂非线性关系。
- **灵活性**：支持多种损失函数和数据类型。
- **鲁棒性**：对缺失值、异常值不敏感。
- **可解释性**：提供特征重要性评估。

局限性

- **训练耗时**：串行迭代难以并行化。

优势与局限性

优势

- **高精度**：处理复杂非线性关系。
- **灵活性**：支持多种损失函数和数据类型。
- **鲁棒性**：对缺失值、异常值不敏感。
- **可解释性**：提供特征重要性评估。

局限性

- **训练耗时**：串行迭代难以并行化。
- **参数敏感**：调优复杂度高。

优势与局限性

优势

- **高精度**：处理复杂非线性关系。
- **灵活性**：支持多种损失函数和数据类型。
- **鲁棒性**：对缺失值、异常值不敏感。
- **可解释性**：提供特征重要性评估。

局限性

- **训练耗时**：串行迭代难以并行化。
- **参数敏感**：调优复杂度高。
- **内存消耗**：存储多棵树资源需求大。

优势与局限性

优势

- **高精度**：处理复杂非线性关系。
- **灵活性**：支持多种损失函数和数据类型。
- **鲁棒性**：对缺失值、异常值不敏感。
- **可解释性**：提供特征重要性评估。
  - **与随机森林对比**：GBRT 降低偏差，随机森林降低方差；GBRT 串行训练，随机森林可并行。

局限性

- **训练耗时**：串行迭代难以并行化。
- **参数敏感**：调优复杂度高。
- **内存消耗**：存储多棵树资源需求大。

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
  - 决策树
  - 集成学习与随机森林
  - AdaBoost
  - Gradient Boosting
  - XGBoost
  - LightGBM
  - 核方法

XGBoost 的诞生与核心思想

- **提出时间**：2014 年由陈天奇提出

XGBoost 的诞生与核心思想

- **提出时间**：2014 年由陈天奇提出
- **核心目标**：在 GBDT 基础上进行系统优化，提升精度和速度

XGBoost 的诞生与核心思想

- **提出时间**：2014 年由陈天奇提出
- **核心目标**：在 GBDT 基础上进行系统优化，提升精度和速度
- **关键改进**：正则化目标函数、二阶泰勒展开、工程优化

XGBoost 的诞生与核心思想

- **提出时间**：2014 年由陈天奇提出
- **核心目标**：在 GBDT 基础上进行系统优化，提升精度和速度
- **关键改进**：正则化目标函数、二阶泰勒展开、工程优化

XGBoost vs GBDT

- GBDT：一阶导数，无正则化，基础实现
- XGBoost：二阶导数，加入正则项，全面优化

XGBoost 的关键机理：正则化目标函数

- **目标函数** = 损失函数 + 正则化项

$$\text{Obj}^{(t)} = \sum_{i=1}^{n} L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

$$\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

- γ：叶子节点分裂阈值
- λ：L2 正则化系数
- T：叶子节点数量
- w_j：叶子节点权重

XGBoost 的关键——二阶泰勒展开

- **二阶近似**：更精确地逼近损失函数

$$\text{Obj}^{(t)} \approx \sum_{i=1}^{n} \left[L(y_i, \hat{y}^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i {f_t}^2(x_i) \right] + \Omega(f_t)$$

- $g_i = \partial_{\hat{y}^{(t-1)}} L(y_i, \hat{y}^{(t-1)})$：一阶梯度
- $h_i = \partial^2_{\hat{y}^{(t-1)}} L(y_i, \hat{y}^{(t-1)})$：二阶梯度

优势

二阶信息提供更准确的方向和步长，收敛更快更稳定

XGBoost 的工程优化

- **块结构**：特征预排序存储，支持并行分裂点查找

XGBoost 的工程优化

- **块结构**：特征预排序存储，支持并行分裂点查找
- **缓存感知**：优化 CPU 缓存访问模式

XGBoost 的工程优化

- **块结构**：特征预排序存储，支持并行分裂点查找
- **缓存感知**：优化 CPU 缓存访问模式
- **外存计算**：处理超大规模数据

XGBoost 的工程优化

- **块结构**：特征预排序存储，支持并行分裂点查找
- **缓存感知**：优化 CPU 缓存访问模式
- **外存计算**：处理超大规模数据

优化技术	GBDT	XGBoost
特征预排序	无	有
并行分裂	无	特征级并行
缺失值处理	需预处理	自动学习分裂方向
正则化	无	L1+L2 正则

XGBoost 的优势与局限

优势

- 预测精度高

XGBoost 的优势与局限

优势

- 预测精度高
- 正则化防止过拟合

XGBoost 的优势与局限

优势

- 预测精度高
- 正则化防止过拟合
- 自动处理缺失值

XGBoost 的优势与局限

优势

- 预测精度高
- 正则化防止过拟合
- 自动处理缺失值
- 丰富的功能支持

XGBoost 的优势与局限

优势

- 预测精度高
- 正则化防止过拟合
- 自动处理缺失值
- 丰富的功能支持

局限

XGBoost 的优势与局限

优势

- 预测精度高
- 正则化防止过拟合
- 自动处理缺失值
- 丰富的功能支持

局限

- 内存消耗较大

XGBoost 的优势与局限

优势

- 预测精度高
- 正则化防止过拟合
- 自动处理缺失值
- 丰富的功能支持

局限

- 内存消耗较大
- 训练速度相对慢

XGBoost 的优势与局限

优势

- 预测精度高
- 正则化防止过拟合
- 自动处理缺失值
- 丰富的功能支持

局限

- 内存消耗较大
- 训练速度相对慢
- 参数调优复杂

XGBoost 的优势与局限

优势

- 预测精度高
- 正则化防止过拟合
- 自动处理缺失值
- 丰富的功能支持

局限

- 内存消耗较大
- 训练速度相对慢
- 参数调优复杂

适用场景

大规模数据集、高精度要求、特征维度高的复杂问题（如金融风控、搜索排序）

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
  - 决策树
  - 集成学习与随机森林
  - AdaBoost
  - Gradient Boosting
  - XGBoost
  - LightGBM
  - 核方法

LightGBM

- **核心目标**：在保证精度的前提下，大幅提升训练速度和减少内存使用

LightGBM

- **核心目标**：在保证精度的前提下，大幅提升训练速度和减少内存使用
- **目标用户**：需要处理大规模数据的应用场景

LightGBM

- **核心目标**：在保证精度的前提下，大幅提升训练速度和减少内存使用
- **目标用户**：需要处理大规模数据的应用场景

LightGBM vs XGBoost

LightGBM

- **核心目标**：在保证精度的前提下，大幅提升训练速度和减少内存使用
- **目标用户**：需要处理大规模数据的应用场景

LightGBM vs XGBoost

- XGBoost：精度优先，全面优化

LightGBM

- **核心目标**：在保证精度的前提下，大幅提升训练速度和减少内存使用
- **目标用户**：需要处理大规模数据的应用场景

LightGBM vs XGBoost

- XGBoost：精度优先，全面优化
- LightGBM：效率优先，针对大数据优化

LightGBM 关键机理：基于梯度的单边采样 (GOSS)

- **核心思想**：保留大梯度样本，随机采样小梯度样本

LightGBM 关键机理：基于梯度的单边采样 (GOSS)

- **核心思想**：保留大梯度样本，随机采样小梯度样本

$$\text{采样权重} = \begin{cases} 1 & \text{对于梯度大的样本} \\ \text{随机采样} & \text{对于梯度小的样本} \end{cases}$$

LightGBM 关键机理：基于梯度的单边采样 (GOSS)

- **核心思想**：保留大梯度样本，随机采样小梯度样本

$$\text{采样权重} = \begin{cases} 1 & \text{对于梯度大的样本} \\ \text{随机采样} & \text{对于梯度小的样本} \end{cases}$$

- **理论依据**：梯度大的样本对信息增益贡献更大

LightGBM 关键机理：基于梯度的单边采样 (GOSS)

- **核心思想**：保留大梯度样本，随机采样小梯度样本

$$\text{采样权重} = \begin{cases} 1 & \text{对于梯度大的样本} \\ \text{随机采样} & \text{对于梯度小的样本} \end{cases}$$

- **理论依据**：梯度大的样本对信息增益贡献更大
- **实践效果**：减少数据量同时保持精度

LightGBM 关键机理：互斥特征捆绑 (EFB)

- **问题背景**：高维特征往往稀疏且互斥。

LightGBM 关键机理：互斥特征捆绑 (EFB)

- **问题背景**：高维特征往往稀疏且互斥。
  - 高维空间的几何本质—— 体积向角落聚集、 球体体积收缩——高维空间中的数据天生就具有稀疏性 。

LightGBM 关键机理：互斥特征捆绑 (EFB)

- **问题背景**：高维特征往往稀疏且互斥。
  - 高维空间的几何本质—— 体积向角落聚集、 球体体积收缩——高维空间中的数据天生就具有稀疏性 。
  - 数据生成过程导致了稀疏性——

LightGBM 关键机理：互斥特征捆绑 (EFB)

- **问题背景**：高维特征往往稀疏且互斥。
  - 高维空间的几何本质—— 体积向角落聚集、 球体体积收缩——高维空间中的数据天生就具有稀疏性 。
  - 数据生成过程导致了稀疏性——
    - 用户-物品交互矩阵 99.9% 以上的位置的值都是 0。

LightGBM 关键机理：互斥特征捆绑 (EFB)

- **问题背景**：高维特征往往稀疏且互斥。
  - 高维空间的几何本质—— 体积向角落聚集、 球体体积收缩——高维空间中的数据天生就具有稀疏性 。
  - 数据生成过程导致了稀疏性——
    - 用户-物品交互矩阵 99.9% 以上的位置的值都是 0。
    - 一篇具体的文档通常只包含其中的几十到几百个词，其余维度上的值均为 0。

LightGBM 关键机理：互斥特征捆绑 (EFB)

- **问题背景**：高维特征往往稀疏且互斥。
  - 高维空间的几何本质—— 体积向角落聚集、 球体体积收缩——高维空间中的数据天生就具有稀疏性 。
  - 数据生成过程导致了稀疏性——
    - 用户-物品交互矩阵 99.9% 以上的位置的值都是 0。
    - 一篇具体的文档通常只包含其中的几十到几百个词，其余维度上的值均为 0。
    - 任何一个具体的事件、样本或实例，都只与所有可能特征中的极小一部分相关。大部分特征对于该样本来说是不适用的或无意义的，因此其值自然为零，从而形成了稀疏向量。

LightGBM 关键机理：互斥特征捆绑 (EFB)

- **问题背景**：高维特征往往稀疏且互斥。
- **解决方案**：将互斥特征捆绑成一个新特征
  -
  - 特征捆绑条件：冲突比率 $<$ 阈值
- **优势**：减少特征维度，降低计算复杂度

实际效果

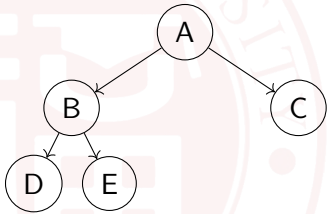
在稀疏特征场景下，特征数量减少 30-50%，训练速度提升 2-3 倍

LightGBM 关键机理：Leaf-wise 生长策略

- **传统方法**：Level-wise，按层生长

LightGBM 关键机理：Leaf-wise 生长策略

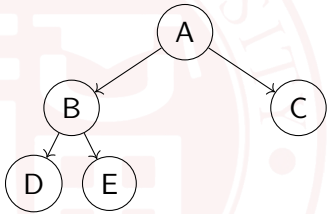
- **传统方法**：Level-wise，按层生长
- **LightGBM**：Leaf-wise，按最大增益生长



Leaf-wise 生长

LightGBM 关键机理：Leaf-wise 生长策略

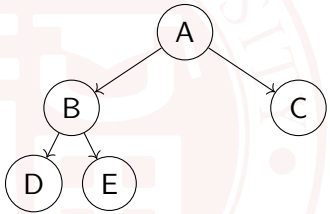
- **传统方法**：Level-wise，按层生长
- **LightGBM**：Leaf-wise，按最大增益生长
- **优势**：更快收敛，更高精度



Leaf-wise 生长

LightGBM 关键机理：Leaf-wise 生长策略

- **传统方法**：Level-wise，按层生长
- **LightGBM**：Leaf-wise，按最大增益生长
- **优势**：更快收敛，更高精度
- **风险控制**：通过 max_depth 参数防止过拟合



Leaf-wise 生长

LightGBM 的直方图算法

- **传统方法**：预排序，精确分裂点查找
- **LightGBM**：直方图近似，离散化查找

$$\text{连续值} \xrightarrow{\text{离散化}} \text{直方图 bin} \xrightarrow{\text{近似分裂}} \text{最优分裂点}$$

- **优势**：内存使用减少 80%，训练速度提升 2-6 倍

算法	内存使用	训练速度
XGBoost 预排序	100%	1x
LightGBM 直方图	20%	2-6x

LightGBM 的优势与局限

优势

- 训练速度极快

LightGBM 的优势与局限

优势

- 训练速度极快
- 内存消耗低

LightGBM 的优势与局限

优势

- 训练速度极快
- 内存消耗低
- 支持类别特征

LightGBM 的优势与局限

优势

- 训练速度极快
- 内存消耗低
- 支持类别特征
- 大数据集表现优异

LightGBM 的优势与局限

优势

- 训练速度极快
- 内存消耗低
- 支持类别特征
- 大数据集表现优异

局限

LightGBM 的优势与局限

优势

- 训练速度极快
- 内存消耗低
- 支持类别特征
- 大数据集表现优异

局限

- 小数据集易过拟合

LightGBM 的优势与局限

优势

- 训练速度极快
- 内存消耗低
- 支持类别特征
- 大数据集表现优异

局限

- 小数据集易过拟合
- 参数敏感性高

LightGBM 的优势与局限

优势

- 训练速度极快
- 内存消耗低
- 支持类别特征
- 大数据集表现优异

局限

- 小数据集易过拟合
- 参数敏感性高
- 理论分析相对复杂

LightGBM 的优势与局限

优势

- 训练速度极快
- 内存消耗低
- 支持类别特征
- 大数据集表现优异

局限

- 小数据集易过拟合
- 参数敏感性高
- 理论分析相对复杂

适用场景

超大规模数据（亿级样本）、高维稀疏特征、实时性要求高的场景（如推荐系统）

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
  - 决策树
  - 集成学习与随机森林
  - AdaBoost
  - Gradient Boosting
  - XGBoost
  - LightGBM
- 核方法

什么是核方法?

- 核方法是一类处理非线性模式分析的技术，属于模式识别算法

什么是核方法?

- 核方法是一类处理非线性模式分析的技术，属于模式识别算法
- 核心思想：通过核函数将数据从原始空间非线性映射到高维特征空间

什么是核方法?

- 核方法是一类处理非线性模式分析的技术，属于模式识别算法
- 核心思想：通过核函数将数据从原始空间非线性映射到高维特征空间
- 在高维空间中，原本线性不可分的数据可能变为线性可分

什么是核方法?

- 核方法是一类处理非线性模式分析的技术，属于模式识别算法
- 核心思想：通过核函数将数据从原始空间非线性映射到高维特征空间
- 在高维空间中，原本线性不可分的数据可能变为线性可分
- 典型应用包括支持向量机、核主成分分析、高斯过程等

什么是核方法?

- 核方法是一类处理非线性模式分析的技术，属于模式识别算法
- 核心思想：通过核函数将数据从原始空间非线性映射到高维特征空间
- 在高维空间中，原本线性不可分的数据可能变为线性可分
- 典型应用包括支持向量机、核主成分分析、高斯过程等

核方法的基本思路

什么是核方法?

- 核方法是一类处理非线性模式分析的技术，属于模式识别算法
- 核心思想：通过核函数将数据从原始空间非线性映射到高维特征空间
- 在高维空间中，原本线性不可分的数据可能变为线性可分
- 典型应用包括支持向量机、核主成分分析、高斯过程等

核方法的基本思路

- 假设：低维空间中线性不可分的点集，在高维空间中可能线性可分

什么是核方法?

- 核方法是一类处理非线性模式分析的技术，属于模式识别算法
- 核心思想：通过核函数将数据从原始空间非线性映射到高维特征空间
- 在高维空间中，原本线性不可分的数据可能变为线性可分
- 典型应用包括支持向量机、核主成分分析、高斯过程等

核方法的基本思路

- 假设：低维空间中线性不可分的点集，在高维空间中可能线性可分
  - 一维不可分数据通过 $F(x) = (x - a)(x - b)$ 映射到二维空间变为可分

什么是核方法?

- 核方法是一类处理非线性模式分析的技术，属于模式识别算法
- 核心思想：通过核函数将数据从原始空间非线性映射到高维特征空间
- 在高维空间中，原本线性不可分的数据可能变为线性可分
- 典型应用包括支持向量机、核主成分分析、高斯过程等

核方法的基本思路

- 假设：低维空间中线性不可分的点集，在高维空间中可能线性可分
  - 一维不可分数据通过 $F(x) = (x - a)(x - b)$ 映射到二维空间变为可分
- 关键优势：无需显式计算高维映射，通过核函数隐式计算高维空间内积

什么是核方法?

- 核方法是一类处理非线性模式分析的技术，属于模式识别算法
- 核心思想：通过核函数将数据从原始空间非线性映射到高维特征空间
- 在高维空间中，原本线性不可分的数据可能变为线性可分
- 典型应用包括支持向量机、核主成分分析、高斯过程等

核方法的基本思路

- 假设：低维空间中线性不可分的点集，在高维空间中可能线性可分
  - 一维不可分数据通过 $F(x) = (x - a)(x - b)$ 映射到二维空间变为可分
- 关键优势：无需显式计算高维映射，通过核函数隐式计算高维空间内积
- 核心机制：通过核技巧避免显式特征变换引发的维度灾难

核方法的起源与发展

- 1964 年：Aizerman 等在势函数方法中首次引入核技术

核方法的起源与发展

- 1964 年：Aizerman 等在势函数方法中首次引入核技术
- 1992 年：Vapnik 等将核技术成功应用于非线性支持向量机

核方法的起源与发展

- 1964 年：Aizerman 等在势函数方法中首次引入核技术
- 1992 年：Vapnik 等将核技术成功应用于非线性支持向量机
- 理论基石：Mercer 定理可追溯至 1909 年

核方法的起源与发展

- 1964 年：Aizerman 等在势函数方法中首次引入核技术
- 1992 年：Vapnik 等将核技术成功应用于非线性支持向量机
- 理论基石：Mercer 定理可追溯至 1909 年
- 再生核希尔伯特空间研究始于 20 世纪 40 年代

核方法的起源与发展

- 1964 年：Aizerman 等在势函数方法中首次引入核技术
- 1992 年：Vapnik 等将核技术成功应用于非线性支持向量机
- 理论基石：Mercer 定理可追溯至 1909 年
- 再生核希尔伯特空间研究始于 20 世纪 40 年代
- 1990 年代后：核方法成为机器学习关键技术类群

核技巧

- 避免显式高维映射：不直接计算高维特征向量 $\phi(x)$

核技巧

- 避免显式高维映射：不直接计算高维特征向量 $\phi(x)$
- 内积计算：通过核函数 $K(x, x') = \langle \phi(x), \phi(x') \rangle$ 直接计算内积

核技巧

- 避免显式高维映射：不直接计算高维特征向量 $\phi(x)$
- 内积计算：通过核函数 $K(x, x') = \langle \phi(x), \phi(x') \rangle$ 直接计算内积
- 维度无关性：计算复杂度与高维特征空间的维数无关

核技巧

- 避免显式高维映射：不直接计算高维特征向量 $\phi(x)$
- 内积计算：通过核函数 $K(x, x') = \langle \phi(x), \phi(x') \rangle$ 直接计算内积
- 维度无关性：计算复杂度与高维特征空间的维数无关
- 关键创新：隐式处理高维空间运算，避免维度灾难

核技巧的数学表达

- 设 $\phi(x)$ 为原始空间到特征空间的映射
- 核函数定义为：$K(x, x') = \phi(x)^\top \phi(x')$
- 实际计算：直接在原始空间通过核函数计算，避免显式映射
- 示例：$K(x, z) = (x^\top z)^2$ 对应特定的特征映射

核方法的主要作用

- 处理非线性问题：将非线性问题转化为高维空间中的线性问题

核方法的主要作用

- 处理非线性问题：将非线性问题转化为高维空间中的线性问题
- 特征提取：自动学习数据的复杂特征表示

核方法的主要作用

- 处理非线性问题：将非线性问题转化为高维空间中的线性问题
- 特征提取：自动学习数据的复杂特征表示
- 适用范围广：可处理向量、序列、图结构等复杂数据

核方法的主要作用

- 处理非线性问题：将非线性问题转化为高维空间中的线性问题
- 特征提取：自动学习数据的复杂特征表示
- 适用范围广：可处理向量、序列、图结构等复杂数据
- 算法扩展：将线性算法扩展至非线性领域

核方法的独特优势

- 避免维度灾难：核技巧使计算复杂度与特征空间维数无关
- 灵活集成先验知识：通过核函数设计融入领域知识
- 强泛化能力：线性学习器有更好的过拟合控制
- 模块化设计：核函数设计与算法设计可分离
- 理论保证：基于统计学习理论和再生核希尔伯特空间

核函数的定义与要求

- 有效核函数：需满足 Mercer 条件（半正定性质）
- 核矩阵：对于任意数据集，Gram 矩阵必须半正定
- 对称性：核函数必须对称，即 $K(x, x') = K(x', x)$
- 构造方法：可通过简单核函数组合构造新核函数

线性核函数

- 公式：$K(x, x') = x^\top x'$
- 特点：最简单核函数，无额外参数
- 适用场景：线性可分或近似线性可分的数据
- 优势：计算简单，不易过拟合

多项式核函数

- 公式：$K(x, x') = (x^\top x' + c)^d$
- 参数：d 为多项式次数，c 为常数项
- 特点：全局核函数，所有点都影响结果
- 适用场景：图像处理、文本分类等

高斯径向基核函数

- 公式：$K(x, x') = \exp(-\frac{\|x-x'\|^2}{2\sigma^2})$
- 参数：σ 控制函数的宽度
- 特点：局部性强，外推能力随参数增大而减弱
- 优势：可将数据映射到无穷维特征空间

其他核函数

- Sigmoid 核：$K(x, x') = \tanh(\beta x^\top x' + b)$，类似神经网络激活函数
- 傅里叶核：适用于信号处理问题
- 样条核：基于样条函数的核函数
- 复合核：通过简单核组合构建的复杂核函数

支持向量机中的核方法

- 核心作用：将线性 SVM 扩展为非线性分类器
- 实现方式：通过核函数计算高维空间中的内积
- 优化问题：对偶形式中仅涉及内积运算，适合核技巧
- 优势：无需知道特征映射的具体形式

SVM 核方法示例

- 原始问题：在输入空间中寻找最优超平面
- 核化版本：在特征空间中通过核函数寻找线性超平面
- 决策函数：$f(x) = \text{sign}(\sum_{i=1}^{N} \alpha_i y_i K(x_i, x) + b)$[1]
- 支持向量：仅支持向量参与决策函数计算

[1] $f(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b\right)$

核主成分分析

- 目标：执行非线性降维
- 实现：在特征空间中进行标准 PCA 分析
- 优势：能够发现数据中的非线性结构
- 应用：特征提取、数据可视化等

高斯过程

- 本质：贝叶斯核方法
- 核心：通过核函数定义数据点之间的协方差
- 应用：回归、分类、不确定性估计
- 优势：提供预测的不确定性度量

其他核化算法

- 核 Fisher 判别分析：非线性判别分析
- 核聚类：非线性聚类算法
- 核回归：非线性回归方法
- 核感知器：感知器的核化版本

核方法与深度学习的结合

- 深度核学习：结合神经网络与高斯过程
- 核化神经网络层：将传统卷积核替换为核函数
- 注意力机制：自注意力中的相似度计算可视为核函数应用
- 优势：小样本学习、不确定性估计

高效核近似方法

- 随机傅里叶特征：通过随机特征映射近似核函数
- Nyström 方法：通过子采样近似核矩阵
- 目标：降低核矩阵计算复杂度，适应大规模数据
- 应用：大规模机器学习任务

结构化核函数学习

- 自适应核学习：在训练过程中自适应调整核参数
- 多核学习：组合多个核函数以获得更好性能
- 深度核函数：通过深度学习自动学习核函数
- 优势：提升模型表达能力

核方法的挑战与局限性

- 核函数选择：依赖先验知识或交叉验证
- 计算复杂度：核矩阵存储和计算复杂度为 $O(N^2)$
- 可解释性：高维特征空间中的决策过程不易解释
- 参数调优：核函数参数对性能影响显著

总结

- 核方法是处理非线性问题的强大工具
- 核技巧通过隐式高维映射避免维度灾难
- 在 SVM、核 PCA 等多个领域有成功应用
- 与深度学习结合是未来重要方向
- 理论基础坚实，实用价值显著

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
- 时间序列模型
  - ARIMA 模型
  - VAR 模型
  - Prophet 模型
  - 模型比较与新发展

时间序列分析基础

- **时间序列定义**：按时间顺序排列的一系列观测值
- **主要特点**：
  - 时序性：数据点之间存在时间依赖关系
  - 趋势性：长期上升或下降的趋势
  - 季节性：周期性变化规律
  - 不确定性：受多种因素影响，难以精确预测
- **分类**：
  - 平稳 vs 非平稳时间序列
  - 线性 vs 非线性时间序列

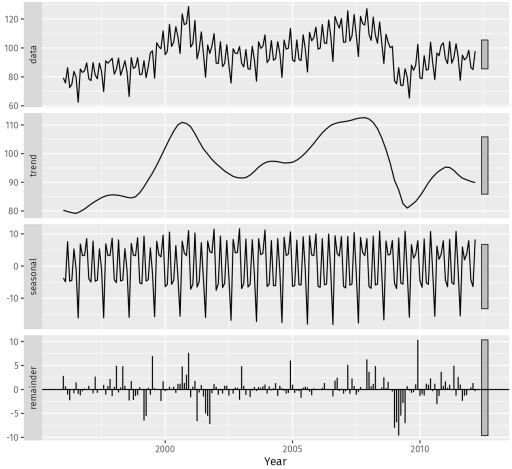
1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
- 时间序列模型
  - ARIMA 模型
  - VAR 模型
  - Prophet 模型
  - 模型比较与新发展

时间序列的趋势、季节性和随机成分



ARIMA

- **起源**：由 Box 和 Jenkins 在 1970 年代系统提出
- **要解决的问题**：如何系统分析和预测**非平稳**单变量时间序列
- **前身**：自回归模型（Auto Regression）和移动平均模型（Moving Averaging）
- **核心贡献**：引入差分使非平稳序列平稳，提供系统方法论

ARIMA 模型原理

ARIMA(p, d, q) 模型由三部分组成：

自回归（AR）部分

$$y_t = c + \sum_{i=1}^{p} \phi_i y_{t-i} + \epsilon_t$$

用自身过去值预测未来，阶数 p 表示用过去几期的值

差分 (I) 部分

一阶差分：$\Delta y_t = y_t - y_{t-1}$，$d$ 表示差分次数

移动平均 (MA) 部分

$$y_t = c + \epsilon_t + \sum_{i=1}^{q} \theta_i \epsilon_{t-i}$$

用历史预测误差改进模型，阶数 q 表示误差项阶数

平稳性检验与参数确定

平稳性检验

- ADF 检验：p 值 >0.05 表示非平稳
- 差分直至平稳，确定 d 值
- 示例：股票价格序列通常需差分

参数识别

- ACF 图：识别 MA 阶数 q
- PACF 图：识别 AR 阶数 p
- 信息准则：AIC/BIC 辅助定阶

ACF/PACF 判据总结

模型类型	ACF	PACF
AR(p)	拖尾	p 阶后截尾
MA(q)	q 阶后截尾	拖尾
ARMA(p,q)	拖尾	拖尾

ARIMA 模型评估与诊断

残差分析：

- 残差应为白噪声（均值为 0，方差恒定）
- Ljung-Box 检验：p 值 >0.05 说明残差无自相关
- ACF/PACF 图检验：残差应无显著自相关

模型比较指标：

- AIC（赤池信息准则）：越小越好，考虑模型复杂度
- BIC（贝叶斯信息准则）：类似 AIC，对参数惩罚更重

ARIMA 局限性：

- 仅适用于单变量序列
- 假设线性关系
- 对季节性数据需使用 SARIMA 扩展

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
- 时间序列模型
  - ARIMA 模型
  - VAR 模型
  - Prophet 模型
  - 模型比较与新发展

VAR 模型：发展背景

- **起源**：Sims 在 1980 年提出，用于宏观经济分析
- **要解决的问题**：分析多变量经济系统内部的动态交互关系
- **核心思想**：将每个变量表示为系统中所有变量滞后值的函数
- **与 ARIMA 区别**：多变量 vs 单变量，系统分析 vs 预测

VAR 模型原理

基本形式

VAR(p) 模型将每个内生变量表示为系统中所有其他变量 p 期滞后值的线性函数：

$$Y_t = c + A_1 Y_{t-1} + A_2 Y_{t-2} + \cdots + A_p Y_{t-p} + e_t$$

其中 Y_t 为向量，包含所有变量在 t 时刻的值

二元 VAR(1) 示例

$$y_{1,t} = c_1 + a_{11} y_{1,t-1} + a_{12} y_{2,t-1} + e_{1,t}$$
$$y_{2,t} = c_2 + a_{21} y_{1,t-1} + a_{22} y_{2,t-1} + e_{2,t}$$

特点：不预设因果关系，平等对待所有变量

VAR 模型建模与应用

建模步骤：

1. 平稳性检验：多变量序列需联合平稳
2. 确定滞后阶数 p：使用信息准则（AIC/BIC）
3. 参数估计：最小二乘法或最大似然法
4. 模型诊断：残差检验、稳定性检验

高级应用：

- 脉冲响应函数：分析变量受到冲击后的动态影响路径
- 方差分解：将预测误差方差归因于不同变量的冲击
- 格兰杰因果检验：分析变量间的领先滞后关系

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
- 时间序列模型
  - ARIMA 模型
  - VAR 模型
  - Prophet 模型
  - 模型比较与新发展

Prophet 模型：发展背景

- **起源**：Facebook 于 2017 年开源发布
- **要解决的问题**：商业时间序列预测的自动化、鲁棒性和可解释性
- **设计理念**：为商业分析者设计，无需时间序列专家也能使用
- **核心思想**：将时间序列分解为趋势、季节性和节假日三个主要组成部分

Prophet 模型原理

加性模型形式

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

- **趋势项** $g(t)$：捕捉非周期性长期变化
  - 分段线性模型或饱和增长模型（考虑承载能力）
- **季节项** $s(t)$：使用傅里叶级数拟合周期性模式
- **节假日项** $h(t)$：模拟不规则日期的重要事件
- **误差项** ϵ_t：模型未解释的随机波动

Prophet 模型优势与参数调整

主要优势：

- 对缺失值和异常值不敏感
- 处理季节性模式变化灵活
- 内置节假日效应处理
- 拟合速度快，自动化程度高

参数调整：

- 趋势灵活度：`changepoint_prior_scale`
- 季节效应强度：`seasonality_prior_scale`
- 节假日影响强度：`holidays_prior_scale`
- 傅里叶项数：控制季节模式平滑度

1 数据挖掘与机器学习的基本概念

2 数据挖掘的数学基础

3 特征工程与预处理

4 传统机器学习模型
- 线性模型
- 非线性模型
- 时间序列模型
  - ARIMA 模型
  - VAR 模型
  - Prophet 模型
- 模型比较与新发展

模型比较与选择指南

特性	ARIMA	VAR	Prophet
核心思想	单变量线性预测	多变量动态系统	可分解加性模型
数据要求	平稳/可差分平稳	多变量平稳	对缺失值/异常值鲁棒
参数调整	ACF/PACF 定阶	信息准则定阶	直观参数调整
优势	理论成熟、简单高效	捕捉变量间动态关系	自动化、可解释性强
局限	仅处理线性关系、单变量	参数多、样本量要求高	主要捕捉加性效应
适用场景	销售预测、需求规划	宏观经济系统分析	商业指标预测

表 1: 时间序列模型比较

时间序列模型新发展

- **混合时间序列模型**
  - 结合多种模型的优点，提高预测精度
  - 如 ARIMA 与神经网络的组合
- **非线性时间序列模型**
  - 处理更复杂的动态模式
  - 如门限自回归 (TAR)、马尔可夫转换模型
- **基于机器学习的方法**
  - 支持向量机 (SVM)、随机森林
  - 擅长捕捉复杂非线性关系
- **深度学习模型**
  - LSTM：擅长捕捉长期依赖关系
  - Transformer：自注意力机制处理长序列
  - 需要大量数据和计算资源

本节课主要内容总结

核心知识体系

- **数据挖掘基本概念**：KDD 定义与流程、机器学习类型（监督/无监督/半监督/强化学习）
- **数学基础**：概率统计、线性代数（SVD、PCA）、微积分与优化理论（梯度下降）
- **特征工程**：特征缩放、数据集分割、缺失值处理、特征选择与降维
- **传统机器学习模型**：线性回归、正则化方法、逻辑回归、支持向量机、决策树

关键技术要点

- 数据集分割方法：留出法、K 折交叉验证、分层交叉验证、留一法
- 特征缩放技术：标准化、最小-最大缩放、Robust 缩放、L2 缩放
- 正则化方法：岭回归（L2）、Lasso 回归（L1）的核心区别与应用场景
- 分类算法：逻辑回归的 Sigmoid 函数与多分类策略（OvR、OvO）

实践应用价值

- 为时空数据挖掘提供坚实的理论基础和方法论支持
- 掌握从数据预处理到模型构建的完整机器学习流程
- 培养解决实际问题的数据分析和建模能力